Minimizing weighted makespan in a two-machine flow shop

Author One[a,*], Author Two[b]

[a]Institution One, Location, Country

[b]Institution Two, Location, Country

Abstract

Motivated by the scheduling of hot lots in semiconductor wafer fabrication, this paper studies the two-machine flow shop problem with the maximum weighted completion time objective, also known as the weighted makespan. We establish binary NP-hardness even with two distinct weights and strong NP-hardness when the number of distinct weights is arbitrary. We further prove that an optimal schedule has a block structure, with one block per distinct weight, blocks arranged in non-increasing order of weight, and jobs within each block sequenced according to Johnson's rule. Exploiting this structure, we develop an exact dynamic programming algorithm that is pseudo-polynomial for a fixed number of distinct weights, a fully polynomial-time approximation scheme for a fixed number of distinct weights, and a polynomial-time approximation scheme when the ratio of the largest to the smallest weight is bounded. We also propose a near-linear-time two-approximation algorithm and adapt the NEH heuristic and a weight-block genetic algorithm (WBGA) for larger instances. Computational experiments on 440 small-scale and 640 large-scale instances are conducted using a mixed-integer programming model and a lower bound as benchmarks. The results show that the two-approximation algorithm is the fastest method and remains closest to the lower bound on large instances. NEH offers the best quality-cost trade-off on small instances, whereas WBGA achieves the best solution quality on large instances at a substantially higher computational cost.

Keywords: Scheduling; Two-machine flow shop; Dynamic programming; Weighted makespan;

*Corresponding author. E-mail address: author@email.com

1 Introduction

Scheduling plays a critical role in semiconductor wafer fabrication, where each wafer lot undergoes hundreds of processing operations and its completion time determines the delivery time of the corresponding customer order. In practice, high-priority hot lots must be completed ahead of regular lots, and their priority weights may vary across customers and processing layers. Scheduling decisions should therefore not only improve overall production performance but also prevent excessive delays to high-priority lots. Two relevant scheduling objectives are the total weighted completion time, which minimizes the sum of weighted completion times, and the maximum weighted completion time, which minimizes the largest weighted completion time. Unlike the conventional makespan, the maximum weighted completion time accounts for differences in lot priorities and focuses on the worst-case weighted completion performance. When all lots have unit weights, it reduces to the conventional makespan.

Research on the maximum weighted completion time has primarily focused on single-machine and parallel-machine environments. For single-machine scheduling, approximation algorithms and polynomial-time approximation schemes have been developed in two-agent settings (Nong et al., 2011), while online variants have been extensively investigated, with competitive algorithms established for both preemptive and non-preemptive settings (Li, 2015). Online algorithms with best-possible guarantees were subsequently developed (Chai et al., 2018). Online variants have since been extended to no-delayed-processing constraints (Li and Liu, 2023), unexpected machine breakdowns (Guo et al., 2024), and a single restart of the jobs (Liang et al., 2024). Later studies have considered release dates, preemption, and multiple objectives in multi-agent scheduling (Yuan et al., 2020), lot scheduling with the maximum weighted tardiness objective (Mosheiov and Sarig, 2023), and identical parallel machines (Sun, 2024). More recent work has addressed order acceptance and rejection decisions (Lu et al., 2025), lot scheduling under the same objective (Zheng et al., 2025), online scheduling on an unbounded parallel-batch machine (Zhang et al., 2025), online settings under (dis)agreeability conditions (Fan et al., 2026), and scheduling with restricted rate- or weight-modifying activities (Zuo et al., 2026a,b). Despite these advances, the maximum weighted completion time objective has received limited attention in flow-shop environments, particularly in the two-machine setting. This gap is practically relevant to semiconductor manufacturing, where delays in high-priority lots may postpone the delivery of entire customer orders. Motivated by this application, we investigate the two-machine flow-shop scheduling problem with the objective of minimizing the maximum weighted completion time.

The two-machine flow shop has been extensively studied since Johnson established his rule for makespan minimization (Johnson, 1954). Subsequent research has developed constructive heuristics for various flow-shop settings (Shao et al., 2021), while the corre-

sponding three-machine problem remains strongly $\mathcal{NP}$-hard (Pinedo, 2022). Other studies have investigated disruptions (Hnaien and Arbaoui, 2023), branch-and-bound algorithms (Liu and Urgo, 2024a), reentrant production (Liu and Urgo, 2024b), alternative objective functions (Shabtay and Gerstl, 2024), and distributed manufacturing environments (Tristan Becker, 2025). Meanwhile, refinements of Johnson's rule continue to attract research interest (Della Croce and Schau, 2026).

The two-machine flow shop provides a basic model of serial production, in which each lot passes through two consecutive processing stages. Such production structures arise in various industries, including semiconductor manufacturing, where lots undergo lithography followed by etching, and textile manufacturing, where dyeing precedes drying.

The rest of the paper is organized as follows. Section 2 fixes the notation, states the completion-time recurrence and gives the mixed integer programming formulation. Section 3 establishes the complexity of the problem. Section 4 proves the structural properties, develops the exact dynamic program, and presents the approximation schemes. Section 5 describes the 2-approximation and the two heuristics. Section 6 reports the computational study, and Section 7 concludes.

2 Problem description

Consider a set $\mathcal{J} = \{J_1, J_2, \ldots, J_n\}$ of n independent and non-preemptive jobs to be scheduled in a two-machine flow shop with machines M_1 and M_2. All jobs are released at time zero and follow the same route, $M_1 \to M_2$. For $j = 1, 2, \ldots, n$, job J_j is represented by a triple $\Phi_j = (a_j, b_j, w_j)$, where a_j and b_j are its nonnegative integer processing times on M_1 and M_2, respectively, and w_j is a positive weight reflecting its importance. Let K be the number of distinct weights. For a feasible schedule π, let $C_j^1(\pi)$ and $C_j^2(\pi)$ denote the completion times of J_j on M_1 and M_2, respectively; thus $C_j(\pi) = C_j^2(\pi)$ is its completion time. The makespan and the weighted makespan (i.e., the maximum weighted completion time) in π are given by

$$C_{\max}(\pi) = \max_{1 \le j \le n} \{C_j(\pi)\} \quad \text{and} \quad C_{\max}^w(\pi) = \max_{1 \le j \le n} \{w_j C_j(\pi)\},$$

respectively. The objective is to minimize the weighted makespan $C_{\max}^w$. Using the standard 3-field scheduling notation (Pinedo, 2022), this problem is denoted by $F2 \,||\, C_{\max}^w$.

In a two-machine flow shop, a schedule is called a *permutation* schedule if the jobs are processed in the same order on both machines. Johnson (1954) showed that an optimal permutation schedule exists for $F2 \,||\, C_{\max}$. By a similar argument, an optimal permutation schedule also exists for $F2 \,||\, C_{\max}^w$. Hence, we restrict attention to permutation schedules. Let $\pi = (\pi(1), \pi(2), \ldots, \pi(n))$ denote a feasible permutation schedule for $F2 \,||\, C_{\max}^w$, where

$J_{\pi(j)}$ is the j-th job in π. By simple calculation, it follows that

$$C_{\pi(j)} = \max_{1\le i\le j}\Big\{\sum_{h=1}^{i} a_{\pi(h)} + \sum_{h=i}^{j} b_{\pi(h)}\Big\}, \quad j = 1, 2, \ldots, n. \tag{1}$$

Then, the weighted makespan of π is given by

$$C^w_{\max}(\pi) = \max_{1\le j\le n}\{w_{\pi(j)}C_{\pi(j)}\}. \tag{2}$$

To formulate the problem as a mixed integer linear program (MILP), we introduce the following decision variables:

- $x_{jk} = 1$ if job J_j is the k-th job, and 0 otherwise;
- $C^1_{[k]}$ is the completion time of the k-th job on M_1;
- $C^2_{[k]}$ is the completion time of the k-th job on M_2;
- C_j is the completion time of job J_j;
- M is a sufficiently large constant (e.g., $M = \sum_{j=1}^{n}(a_j + b_j)$).

The MILP model can be formulated as follows:

$$\min \quad C^w_{\max} \tag{3}$$

$$\text{s.t.} \quad \sum_{k=1}^{n} x_{jk} = 1, \quad j = 1, 2, \ldots, n \tag{4}$$

$$\sum_{j=1}^{n} x_{jk} = 1, \quad k = 1, 2, \ldots, n \tag{5}$$

$$C^1_{[1]} = \sum_{j=1}^{n} a_j x_{j1} \tag{6}$$

$$C^1_{[k]} = C^1_{[k-1]} + \sum_{j=1}^{n} a_j x_{jk}, \quad k = 2, 3, \ldots, n \tag{7}$$

$$C^2_{[1]} = C^1_{[1]} + \sum_{j=1}^{n} b_j x_{j1} \tag{8}$$

$$C^2_{[k]} \ge C^1_{[k]} + \sum_{j=1}^{n} b_j x_{jk}, \quad k = 2, 3, \ldots, n \tag{9}$$

$$C^2_{[k]} \ge C^2_{[k-1]} + \sum_{j=1}^{n} b_j x_{jk}, \quad k = 2, 3, \ldots, n \tag{10}$$

$$C_j \ge C^2_{[k]} - M(1 - x_{jk}), \quad j = 1, 2, \ldots, n; \quad k = 1, 2, \ldots, n \tag{11}$$

$$C^w_{\max} \ge w_j C_j, \quad j = 1, 2, \ldots, n \tag{12}$$

$$x_{jk} \in \{0,1\}, \quad j = 1,2,\ldots,n; \quad k = 1,2,\ldots,n \tag{13}$$

$$C_j, C^1_{[k]}, C^2_{[k]} \geq 0 \quad j = 1,2,\ldots,n; \quad k = 1,2,\ldots,n \tag{14}$$

The objective is to minimize $C^w_{\max}$, as expressed in (3). Constraints (4) and (5) assign each job to exactly one position and each position to exactly one job. Constraints (6) and (7) determine the completion times on M_1, and (8) through (10) determine those those on M_2. In particular, for $k \geq 2$, the job in position k can start on M_2 only after it has completed M_1 and the preceding job has finished on M_2. Constraint (11) relates the position-based completion times to the job-based ones via a big-M formulation. Note that when $x_{jk} = 1$ (i.e., J_j is assigned to the k-th position), it requires $C_j \geq C^2_{[k]}$; when $x_{jk} = 0$, it is inactive, since $C^2_{[k]} - M \leq 0$ and $C_j \geq 0$. Constraint (12) imposes $WC_{\max} \geq w_j C_j$ for each job, so minimizing $C^w_{\max}$ yields the optimal objective value of (2). Constraints (13) and (14) require the decision variables x_{jk} to be binary and the completion times nonnegative.

3 Computational complexity

When $K = 1$ (i.e., all jobs have equal weights), $F2 \mid\mid C^w_{\max}$ reduces to $F2 \mid\mid C_{\max}$, which is solvable in $O(n \log n)$ time (Johnson, 1954). In contrast, We show that $F2 \mid\mid C^w_{\max}$ is binary $\mathcal{NP}$-hard even with only two distinct weights and strongly $\mathcal{NP}$-hard when K is arbitrary.

Theorem 3.1. *$F2 \mid\mid C^w_{\max}$ is binary $\mathcal{NP}$-hard even with only two distinct weights.*

Proof. To establish the result, we give a polynomial-time reduction from PARTITION, which is $\mathcal{NP}$-complete in the ordinary sense (Garey and Johnson, 1979).

PARTITION: Given r positive integers $x_1, x_2, \ldots, x_r$ and an integer X with $\sum_{i=1}^{r} x_i = 2X$, does there exist a partition of $\{1,2,\ldots,r\}$ into two disjoint subsets $\mathcal{X}_1$ and $\mathcal{X}_2$ such that $\sum_{i\in\mathcal{X}_1} x_i = \sum_{i\in\mathcal{X}_2} x_i = X$?

Given an arbitrary instance $(x_1, x_2, \ldots, x_r; X)$ of PARTITION, we construct an instance of $F2 \mid\mid C^w_{\max}$ with $n = r + 1$ jobs. For $j = 1,2,\ldots,n-1$, job J_j has $a_j = 1$, $b_j = rx_j$, and $w_j = X + 1$; job J_n has $a_n = rX$, $b_n = 1$, and $w_n = 2X + 1$. Set the threshold to $Y = r(X+1)(2X+1)$. The question is whether there exists a schedule with $C^w_{\max} \leq Y$.

We demonstrate that $F2 \mid\mid WC_{\max}$ has a feasible schedule π with $C^w_{\max}(\pi) \leq Y$ if and only if PARTITION has a solution.

Suppose that PARTITION admits a solution $(\mathcal{X}_1, \mathcal{X}_2)$ with $\sum_{j\in\mathcal{X}_1} x_j = \sum_{j\in\mathcal{X}_2} x_j = X$. Define $\mathcal{J}_1 = \{J_j : j \in \mathcal{X}_1\}$ and $\mathcal{J}_2 = \{J_{r+j} : j \in \mathcal{X}_2\}$. We build a feasible permutation schedule π in which both machines process the jobs in the order $\mathcal{J}_1 \to J_n \to \mathcal{J}_2$, with the jobs in $\mathcal{J}_1$ and $\mathcal{J}_2$ processed in arbitrary order (see Figure 1).

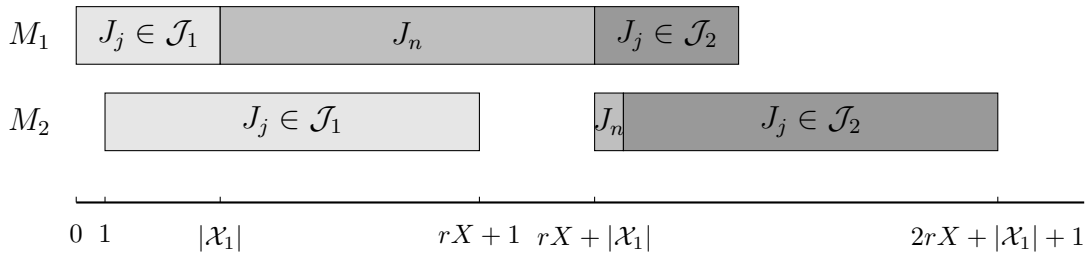


Figure 1: A feasible permutation scheule π

Since $a_j \le b_j$ for each $j = 1, 2, \ldots, r$ and $|\mathcal{X}_1| \le r-1$, it can be easily verified that $C_n(\pi) = rX + |\mathcal{X}_1| + 1 \le r(X+1)$ and $C_{\max}(\pi) = 2rX + |\mathcal{X}_1| + 1 \le r(2X+1)$. Hence, $C^w_{\max}(\pi) \le r(X+1)(2X+1) = Y$.

Conversely, suppose that $F2 \,||\, C^w_{\max}$ admits a feasible schedule π with $C^w_{\max}(\pi) \le Y$. Let $\mathcal{J}_1 = \{J_j : C_j(\pi) < C_n(\pi)\}$ and $\mathcal{J}_2 = \{J_j : C_j(\pi) > C_n(\pi)\}$ be the sets of jobs completed before and after J_n in π, respectively, and let $\mathcal{X}_1 = \{j : J_j \in \mathcal{J}_1\}$ and $\mathcal{X}_2 = \{j : J_j \in \mathcal{J}_2\}$. By (1), we have

$$C_n(\pi) \ge \sum_{j\in\mathcal{X}_1} a_j + a_n + 1 = |\mathcal{X}_1| + rX + 1 \tag{15}$$

and

$$C_n(\pi) \ge a_{\pi(1)} + \sum_{j\in\mathcal{X}_1} b_j + b_n = r\sum_{j\in\mathcal{X}_1} x_j + 2. \tag{16}$$

Case 1. If $\sum_{j\in\mathcal{X}_1} x_j > X$, then $\sum_{j\in\mathcal{X}_1} x_j \ge X+1$ because the x_j are integers. By (16), $WC_{\max}(\pi) \ge w_n C_n(\pi) \ge (2X+1)(r(X+1)+2) > r(X+1)(2X+1) = Y$, contradicting the assumption that $C^w_{\max}(\pi) \le Y$.

Case 2. If $\sum_{j\in\mathcal{X}_1} x_j < X$, then $\sum_{j\in\mathcal{X}_1} x_j \le X-1$ because the x_j are integers. Since $(\mathcal{X}_1, \mathcal{X}_2)$ is a partition of $\{1, 2, \ldots, r\}$, we have

$$\sum_{j\in\mathcal{X}_2} x_j = 2X - \sum_{j\in\mathcal{X}_1} x_j \ge X+1. \tag{17}$$

By (1), (15), and (17), we have

$$C_{\pi(n)}(\pi) = C_{\max}(\pi) \ge C_n(\pi) + \sum_{j\in\mathcal{X}_2} b_j \ge |\mathcal{X}_1| + rX + 1 + r\sum_{j\in\mathcal{X}_2} x_j \ge r(2X+1) + 1. \tag{18}$$

Using $w_{\pi(n)} = 1$, (18) yields

$$C^w_{\max}(\pi) \ge w_{\pi(n)} C_{\pi(n)}(\pi) \ge (X+1)(r(2X+1)+1) > r(X+1)(2X+1) = Y,$$

This contradicts the assumption that $C^w_{\max}(\pi) \le Y$.

Combining the above two cases, we obtain that $\sum_{j\in\mathcal{X}_1} x_j = \sum_{j\in\mathcal{X}_2} x_j = X$. Therefore, $(\mathcal{X}_1, \mathcal{X}_2)$ is a solution to PARTITION. □

Theorem 3.2. *$F2 \mid\mid C^w_{\max}$ is strongly $\mathcal{NP}$-hard when K is arbitrary.*

Proof. To establish the result, we give a polynomial-time reduction from 3-PARTITION, which is strongly $\mathcal{NP}$-complete (Garey and Johnson, 1979).

3-PARTITION: Given $3r$ positive integers $x_1, x_2, \ldots, x_{3r}$ and an integer X with $\sum_{i=1}^{3r} x_i = rX$ and $\frac{X}{4} < x_j < \frac{X}{2}$ for each $j = 1, 2, \ldots, 3r$, does there exist a partition of $\{1, 2, \ldots, 3r\}$ into m disjoint subsets $\mathcal{X}_1, \mathcal{X}_2, \ldots, \mathcal{X}_r$ such that $\sum_{j \in \mathcal{X}_i} x_j = X$ for $i = 1, 2, \ldots, r$?

Given an arbitrary instance $(x_1, x_2, \ldots, x_{3r}; X)$ of 3-PARTITION, we construct an instance of $F2 \mid\mid C^w_{\max}$ consisting of r separating jobs and $3r$ basic jobs. For $j = 1, 2, \ldots, r$, separating job J_j has $a_j = 4X$, $b_j = X$, and $w_j = \frac{5r+4}{5j}$; for $j = r+1, r+2, \ldots, 4r$, basic job J_j has $a_j = x_{j-r}$, $b_j = 4x_{j-r}$, and $w_j = 1$. Set the threshold to $Y = (5r+4)X$. The question is whether there exists a schedule with $WC_{\max} \leq Y$.

We demonstrate that $F2 \mid\mid C^w_{\max}$ has a feasible schedule π with $C^w_{\max}(\pi) \leq Y$ if and only if 3-PARTITION has a solution.

Suppose that 3-PARTITION admits a solution $(\mathcal{X}_1, \mathcal{X}_2, \ldots, \mathcal{X}_r)$, so that $\sum_{j \in \mathcal{X}_i} x_j = X$ and $|\mathcal{X}_i| = 3$ for all i. Let $\mathcal{J}_i = \{J_{r+j} : j \in \mathcal{X}_i\}$ for $i = 1, 2, \ldots, r$. We build a feasible permutation schedule π in which both machines process the jobs in the order

$$J_1 \to \mathcal{J}_1 \to J_2 \to \mathcal{J}_2 \to \cdots \to J_r \to \mathcal{J}_r,$$

with the jobs in each $\mathcal{J}_i$ processed in arbitrary order (see Figure 2).

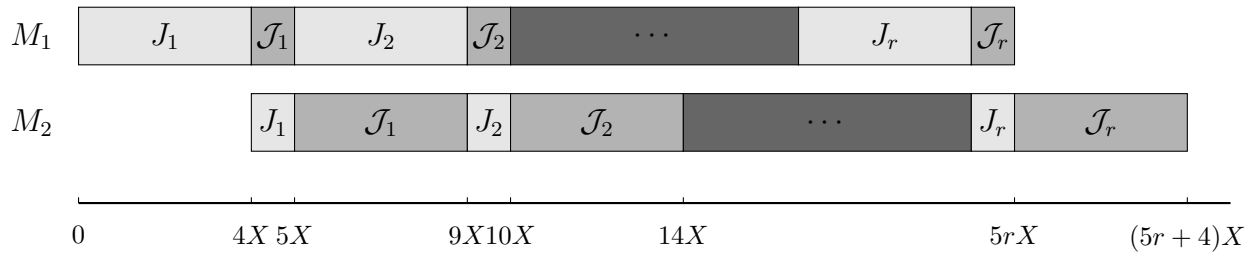


Figure 2: A feasible permutation scheule π

According to Figure 2, it can be easily verified that $C_j(\pi) = 5jX$ for $j = 1, 2, \ldots, r$, and $C_j(\pi) \leq C_{\max}(\pi) = (5r+4)X$ for $j = r+1, r+2, \ldots, 4r$. Then $w_j C_j(\pi) = (5r+4)X = Y$ for $j = 1, 2, \ldots, r$, and $w_j C_j(\pi) \leq w_j C_{\max}(\pi) = (5r+4)X = Y$ for $j = r+1, r+2, \ldots, 4r$. Hence, $C^w_{\max}(\pi) \leq Y$.

Conversely, suppose that $F2 \mid\mid C^w_{\max}$ admits a feasible schedule π with $C^w_{\max}(\pi) \leq Y$. Because all separating jobs have the same processing time on M_1 and M_2 and weights $w_1 > w_2 > \cdots > w_r$, a standard interchange argument lets us assume they appear in π in decreasing order of weight.

Claim 1. $C_{\max}(\pi) \leq Y = (5r+4)X$.

Proof of Claim 1. Since $w_j \geq 1$ for all $1 \leq j \leq 4r$, we have $C_{\max}(\pi) \leq C^w_{\max}(\pi) \leq Y = (5r+4)X$. ■

Claim 2. J_1 is the first job scheduled in π.

Proof of Claim 2. Since $C^w_{\max}(\pi) \leq Y = (5r+4)X$ and $w_1 = \frac{5r+4}{5}$, we have $a_1 + b_1 \leq C_1(\pi) \leq \frac{C^w_{\max}(\pi)}{w_1} \leq 5X = a_1 + b_1$. This means that J_1 is the first job scheduled in π. ■

For $i = 1, 2, \ldots, r-1$, let $\mathcal{B}_i = \{J_j : C_i(\pi) < C_j(\pi) < C_{i+1}(\pi)\}$ be the set of basic jobs scheduled between J_i and J_{i+1} in π, and let $\mathcal{B}_r = \{J_j : C_j(\pi) > C_r(\pi)\}$ be the set of basic jobs scheduled after J_r. For $i = 1, 2, \ldots, r$, define $\mathcal{X}_i = \{j - r : J_j \in \mathcal{B}_i\}$. Then, by Claim 2, π can be written as

$$\pi = (J_1 \to \mathcal{B}_1 \to J_2 \to \mathcal{B}_2 \to \cdots \to J_r \to \mathcal{B}_r). \tag{19}$$

Since $(\mathcal{X}_1, \mathcal{X}_2, \ldots, \mathcal{X}_r)$ is a partition of $\{1, 2, \ldots, 3r\}$, we have

$$\sum_{i=1}^{r} \sum_{j \in \mathcal{X}_i} x_j = \sum_{j=1}^{3r} x_j = rX. \tag{20}$$

Claim 3. For $k = 1, 2, \ldots, r-1$, $\sum_{i=1}^{k} \sum_{j \in \mathcal{X}_i} x_j \leq kX$.

Proof of Claim 3. Suppose to the contrary that there exists some m $(1 \leq m \leq r-1)$ with $\sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j > mX$. Since the x_j are integers, this implies

$$\sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j \geq mX + 1. \tag{21}$$

Because $a_j = 4X$ and $b_j = X$ for $1 \leq j \leq r$, and $a_j = x_{j-r}$ for $r+1 \leq j \leq 4r$, (1), (19), and (21) give

$$C_{m+1}(\pi) \geq \sum_{j=1}^{m+1} a_j + \sum_{i=1}^{m} \sum_{j \in \mathcal{B}_i} a_j + b_{m+1} = 4(m+1)X + \sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j + X \geq 5(m+1)X + 1. \tag{22}$$

Using $w_{m+1} = \frac{5r+4}{5(m+1)}$, (22) yields

$$C^w_{\max}(\pi) \geq w_{m+1} C_{m+1}(\pi) \geq \frac{5r+4}{5(m+1)}(5(m+1)X + 1) > (5r+4)X = Y, \tag{23}$$

which contradicts the assumption that $C^w_{\max}(\pi) \leq Y$. Hence, Claim 3 holds. ■

Claim 4. For $k = 1, 2, \ldots, r-1$, $\sum_{i=1}^{k} \sum_{j \in \mathcal{X}_i} x_j \geq kX$.

Proof of Claim 4. Suppose to the contrary that there exists some m $(1 \leq m \leq r-1)$ with $\sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j < mX$. By (20) and the x_j are integers, we have

$$\sum_{i=m+1}^{r} \sum_{j \in \mathcal{X}_i} x_j = rX - \sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j \geq (r-m)X + 1. \tag{24}$$

Because $a_j = 4X$ and $b_j = X$ for $1 \le j \le r$, and $a_j = x_{j-r}$ and $b_j = 4x_{j-r}$ for $r+1 \le j \le 4r$, (1), (19), (20), and (24) give

$$\begin{aligned} C_{\max}(\pi) &\ge \sum_{j=1}^{m+1} a_j + \sum_{i=1}^{m} \sum_{j \in \mathcal{B}_i} a_j + \sum_{j=m+1}^{r} b_j + \sum_{i=m+1}^{r} \sum_{j \in \mathcal{B}_i} b_j \\ &= 4(m+1)X + \sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j + (r-m)X + \sum_{i=m+1}^{r} \sum_{j \in \mathcal{X}_i} 4x_j \\ &= 4(m+1)X + (r-m)X + rX + 3 \sum_{i=m+1}^{r} \sum_{j \in \mathcal{X}_i} x_j \\ &\ge 4(m+1)X + (r-m)X + rX + 3((r-m)X + 1) \\ &= (5r+4)X + 3 > (5r+4)X = Y. \end{aligned}$$

which contradicts Claim 1. Hence, Claim 4 holds. ■

Combining Claims 3 and 4 with (20), we obtain that $\sum_{j \in \mathcal{X}_i} x_j = X$ for $i = 1, 2, \ldots, r$. Since $X/4 < x_j < X/2$ for each j, a sum equal to X must contain exactly three elements; hence $|\mathcal{X}_i| = 3$ for each i. Therefore, $(\mathcal{X}_1, \mathcal{X}_2, \ldots, \mathcal{X}_r)$ is a solution to 3-Partition. □

4 Fixed number of weights

In this section, we focus on the case where the number of weights K is fixed, i.e., $K = O(1)$. For $K = 1$, the problem is exactly $F2 \,||\, C_{\max}$, which Johnson's rule solves in $O(n \log n)$ time (Johnson, 1954); the rule is summarized in Algorithm 1. For $K = 2$, the problem $F2 \,||\, C^w_{\max}$ is already binary $\mathcal{NP}$-hard (Theorem 3.1). For fixed K, we develop an exact DP algorithm running in pseudo-polynomial time and derive an FPTAS from it.

Algorithm 1: Johnson's rule for $F2 \,||\, C_{\max}$

Input: Job set $\mathcal{J}$; job parameters a_j and b_j for $j = 1, 2, \ldots, n$.

1 **Step 1.** Partition $\mathcal{J}$ into two sets $\mathcal{J}^1 = \{J_j : a_j \le b_j\}$ and $\mathcal{J}^2 = \{J_j : a_j > b_j\}$.

2 **Step 2.** Order the jobs in $\mathcal{J}^1$ in nondecreasing order of a_j, followed by those in $\mathcal{J}^2$ in nondecreasing order of b_j.

3 **Step 3.** Process the jobs in this order on M_1 and then on M_2, yielding the permutation schedule π.

Output: The resulting schedule π.

Let $W_1 > W_2 > \cdots > W_K$ denote the distinct weights of the n jobs, and let $\mathcal{G}_k = \{J_j \in \mathcal{J} : w_j = W_k\}$ be set of jobs having weight W_k, $k = 1, 2, \ldots, K$.

4.1 Block structure of an optimal schedule

To establish the key structural lemma, we first introduce some notation for partitioning a permutation schedule into blocks.

Definition 4.1. ***(Block partition)*** *Let $\pi = (\pi(1), \pi(2), \ldots, \pi(n))$ be a permutation schedule. Set $\ell_0 = 0$ and $\mathcal{R}_1 = \mathcal{J}$. For $k = 1, 2, \ldots, K$, if $\mathcal{R}_k \cap \mathcal{G}_k \neq \emptyset$, define $\ell_k = \max\{j : J_{\pi(j)} \in \mathcal{R}_k \cap \mathcal{G}_k\}$, $\mathcal{B}_k^\pi = \{J_{\pi(\ell_{k-1}+1)}, J_{\pi(\ell_{k-1}+2)}, \ldots, J_{\pi(\ell_k)}\}$, and $\mathcal{R}_{k+1} = \mathcal{J} \setminus \bigcup_{i=1}^{k} \mathcal{B}_i^\pi$. Otherwise, set $\ell_k = \ell_{k-1}$, $\mathcal{B}_k^\pi = \emptyset$, and $\mathcal{R}_{k+1} = \mathcal{R}_k$. Then we call $\mathcal{B}(\pi) = (\mathcal{B}_1^\pi, \mathcal{B}_2^\pi, \ldots, \mathcal{B}_K^\pi)$ the block partition of π.*

Thus, ℓ_k is the position of the last job in $\mathcal{R}_k$ of weight W_k, $\mathcal{B}_k^\pi$ is the k-th block (possibly empty), and $\mathcal{R}_k$ is the set of jobs left after removing the first $k-1$ blocks. By construction, every nonempty block $\mathcal{B}_k^\pi$ ends with a job of weight W_k (i.e., $w_{\pi(\ell_k)} = W_k$), and all jobs in the block have weight at most W_k, i.e., $w_{\pi(j)} \leq W_k$ for all $\ell_{k-1} + 1 \leq j \leq \ell_k$. This means that

$$W_k C_{\pi(\ell_k)}(\pi) = \max\{w_{\pi(j)} C_{\pi(j)}(\pi) : \ell_{k-1} + 1 \leq j \leq \ell_k\}. \tag{25}$$

Consequently, (2) can be reformulated as

$$C_{\max}^w(\pi) = \max_{1 \leq k \leq K} \{W_k C_{\pi(\ell_k)(\pi)}\}. \tag{26}$$

Definition 4.2. ***(Regular block and regular permutation schedule)*** *Let $\pi = (\pi(1), \pi(2), \ldots, \pi(n))$ be a permutation schedule with block partition $\mathcal{B}(\pi) = (\mathcal{B}_1^\pi, \mathcal{B}_2^\pi, \ldots, \mathcal{B}_K^\pi)$. A block $\mathcal{B}_k^\pi$ is called regular if it is empty or if the sequence of its jobs in π matches the sequence obtained by applying Johnson's rule to that block. The schedule π is called regular if all blocks $\mathcal{B}_1^\pi, \mathcal{B}_2^\pi, \ldots, \mathcal{B}_K^\pi$ are regular.*

To illustrate the block partition, we give a small numerical example for $F2 \mid\mid C_{\max}^w$.

Example 4.1. *Consider a six-job instance of $F2 \mid\mid C_{\max}^w$ whose job data are given in Table 1, and let $\pi = (1, 2, \ldots, 6)$ be a permutation schedule.*

Table 1: Job data for Example 4.1

	J_1	J_2	J_3	J_4	J_5	J_6
a_j	2	5	6	8	2	3
b_j	5	3	9	4	3	3
w_j	2	3	5	2	3	2

In Example 4.1, we have $K = 3$, $\mathcal{G}_1 = \{J_3\}$, $\mathcal{G}_2 = \{J_2, J_5\}$, and $\mathcal{G}_3 = \{J_1, J_4, J_6\}$. The resulting blocks are $\mathcal{B}_1^\pi = \{J_1, J_2, J_3\}$, $\mathcal{B}_2^\pi = \{J_4, J_5\}$, and $\mathcal{B}_3^\pi = \{J_6\}$ (see Figure 3). By (26), the weighted makespan of π is $C_{\max}^w(\pi) = \max\{5 \times 22, 3 \times 29, 2 \times 32\} = 110$.

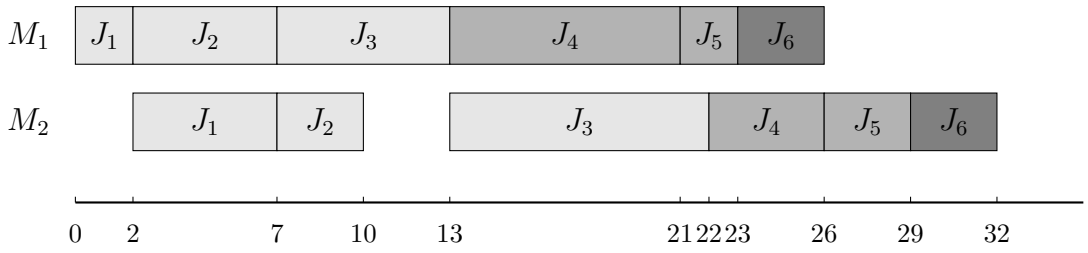


Figure 3: A block partition of π

Given a permutation schedule $\pi = (\pi(1), \pi(2), \ldots, \pi(n))$, for $1 \leq g \leq h \leq n$, define

$$C^w_{\max}(\pi)[g:h] = \max_{g \leq j \leq h} \{w_{\pi(j)} C_{\pi(j)}(\pi)\}$$

as the maximum weighted completion time over the jobs $J_{\pi(g)}, J_{\pi(g+1)}, \ldots, J_{\pi(h)}$.

The following lemma shows that if the jobs in each block of π are rescheduled in accordance with Johnson's rule, the resulting permutation schedule π' has objective value no larger than that of π, i.e., $C^w_{\max}(\pi') \leq C^w_{\max}(\pi)$.

Lemma 4.1. *Given a permutation schedule π and its block partition $\mathcal{B}(\pi) = (\mathcal{B}^\pi_1, \mathcal{B}^\pi_2, \ldots, \mathcal{B}^\pi_K)$, let π_k be the permutation of $\mathcal{B}^\pi_k$ obtained by Johnson's rule for $k = 1, 2, \ldots, K$. Then the schedule π' formed by concatenating $\pi_1, \pi_2, \ldots, \pi_K$ satisfies $C^w_{\max}(\pi') \leq C^w_{\max}(\pi)$.*

Proof. First, if $\mathcal{B}^\pi_k \neq \emptyset$, then rescheduling the jobs in $\mathcal{B}^\pi_k$ according to Johnson's rule does not change the set of jobs assigned to positions $\ell_{k-1}+1, \ell_{k-1}+2, \ldots, \ell_k$, i.e.,

$$\mathcal{B}^\pi_k = \{J_{\pi(\ell_{k-1}+1)}, J_{\pi(\ell_{k-1}+2)}, \ldots, J_{\pi(\ell_k)}\} = \{J_{\pi'(\ell_{k-1}+1)}, J_{\pi'(\ell_{k-1}+2)}, \ldots, J_{\pi'(\ell_k)}\}. \tag{27}$$

Thus, in π', the jobs in $\mathcal{B}^\pi_k$ still occupy this same index interval. By (25), we have

$$C^w_{\max}(\pi)[\ell_{k-1}+1:\ell_k] = W_k C_{\pi(\ell_k)}(\pi) = w_{\pi(\ell_k)} C_{\pi(\ell_k)}(\pi), k = 1, 2, \ldots, K. \tag{28}$$

Moreover, by (27), for $k = 1, 2, \ldots, K$, $\mathcal{B}^\pi_k$ has the same completion time on M_1 under π and π', i.e., $C^1_{\pi'(\ell_k)}(\pi') = C^1_{\pi(\ell_k)}(\pi) = \sum_{j=1}^{\ell_k} a_{\pi(j)}$.

Second, we prove by induction on k that, for all $k = 1, 2, \ldots, K$,

$$C_{\pi'(\ell_k)}(\pi') \leq C_{\pi(\ell_k)}(\pi) \text{ and } C^w_{\max}(\pi')[\ell_{k-1}+1:\ell_k] \leq C^w_{\max}(\pi)[\ell_{k-1}+1:\ell_k]. \tag{29}$$

For the base case $i = 1$, both machines are available at time zero. Rescheduling the jobs in $\mathcal{B}^\pi_1$ according to Johnson's rule gives $C_{\pi'(\ell_1)}(\pi') \leq C_{\pi(\ell_1)}(\pi)$. Since π' only permutes the jobs in $\mathcal{B}^\pi_1$, we have $w_{\pi'(j)} \leq W_1$ for $j = 1, 2, \ldots, \ell_1$. Hence, we have

$$C^w_{\max}(\pi')[1:\ell_1] = \max_{1 \leq j \leq \ell_1} \{w_{\pi'(j)} C_{\pi'(j)}(\pi')\} \leq W_1 C_{\pi'(\ell_1)}(\pi') \leq W_1 C_{\pi(\ell_1)}(\pi) = C^w_{\max}(\pi)[1:\ell_1],$$

where the last equality follows from (28)

Assume that (29) holds for all $i = 1, 2, \ldots, k$. We now prove the $(k+1)$-block. If $\mathcal{B}^{\pi}_{k+1} = \emptyset$, the assertion is immediate. It remains to consider $\mathcal{B}^{\pi}_{k+1} \neq \emptyset$. In this case, in π', by the inductive hypothesis, M_1 becomes available at time $C^1_{\pi'(\ell_k)}(\pi') = C^1_{\pi(\ell_k)}(\pi)$, and M_2 is available at time $C_{\pi'(\ell_k)}(\pi') \leq C_{\pi(\ell_k)}(\pi)$. We introduce a dummy job J_0 with processing times $a_0 = 0$ and $b_0 = C_{\pi'(\ell_k)}(\pi') - C^1_{\pi'(\ell_k)}(\pi')$, and apply Johnson's rule to schedule the jobs in $\mathcal{B}^{\pi}_{k+1} \cup \{J_0\}$, with both machines available from time $C^1_{\pi'(\ell_k)}(\pi')$. Because $a_0 = 0$, J_0 is scheduled in the first position according to Johnson's rule, and it completes at time $C_{\pi'(\ell_k)}(\pi')$. By positioning the dummy job in this way, the jobs in $\mathcal{B}^{\pi}_{k+1}$ can be scheduled on M_2 without overlapping with the previously scheduled jobs in $\mathcal{B}^{\pi}_1 \cup \mathcal{B}^{\pi}_2 \cup \cdots \cup \mathcal{B}^{\pi}_k$. Since the jobs in $\mathcal{B}_{k+1}$ are scheduled according to Johnson's rule, it follows that $C_{\pi'(\ell_{k+1})}(\pi') \leq C_{\pi(\ell_{k+1})}(\pi)$. Moreover, since π' only permutes the jobs in $\mathcal{B}^{\pi}_{k+1}$, we have $w_{\pi'(j)} \leq W_{k+1}$ for $j = \ell_k + 1, \ell_k + 2, \ldots, \ell_{k+1}$. Hence,

$$\begin{aligned} C^w_{\max}(\pi')[\ell_k + 1 : \ell_{k+1}] &= \max_{\ell_k+1 \leq j \leq \ell_{k+1}} \{w_{\pi'(j)} C_{\pi'(j)}(\pi')\} \leq W_{k+1} C_{\pi'(\ell_{k+1})}(\pi') \\ &\leq W_{k+1} C_{\pi(\ell_{k+1})}(\pi) = C^w_{\max}(\pi)[\ell_k + 1 : \ell_{k+1}], \end{aligned}$$

where the last equality follows from (28). This completes the inductive step.

Note that $C^w_{\max}(\pi') = \max_{1 \leq k \leq K} \{C^w_{\max}(\pi')[\ell_{k-1}+1 : \ell_k]\}$ and $C^w_{\max}(\pi) = \max_{1 \leq k \leq K} \{C^w_{\max}(\pi)[\ell_{k-1}+1 : \ell_k]\}$. Hence, (29) yields $C^w_{\max}(\pi') \leq C^w_{\max}(\pi)$. □

To illustrate Lemma 4.1, consider Example 4.1. Given π and its block partition $\mathcal{B}(\pi) = (\mathcal{B}^{\pi}_1, \mathcal{B}^{\pi}_2, \mathcal{B}^{\pi}_3)$, we apply Johnson's rule independently within each block $\mathcal{B}^{\pi}_k$, $k = 1, 2, 3$, obtaining a permutation π_k of the jobs in that block. Concatenating π_1, π_2, π_3 gives $\pi' = (1, 3, 2, 5, 4, 6)$, with blocks $\mathcal{B}^{\pi'}_1 = \{J_1, J_3\}$, $\mathcal{B}^{\pi'}_2 = \{J_2, J_5\}$, and $\mathcal{B}^{\pi'}_3 = \{J_4, J_6\}$ (see Figure 4). By (26), the weighted makespan of π' is $C^w_{\max}(\pi') = \max\{5 \times 17, 3 \times 23, 2 \times 30\} = 85$.

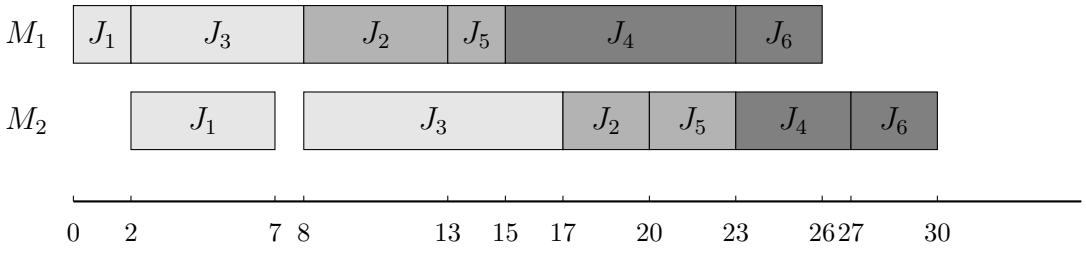


Figure 4: An applicaiton of Lemma 4.1 to π

Lemma 4.2. *For $F2 \,||\, C^w_{\max}$, there exists an optimal permutation schedule that is regular.*

Proof. Let π be an optimal permutation schedule for $F2 \,||\, C^w_{\max}$. If π is regular, the lemma follows. Otherwise, let $\mathcal{B}(\pi) = (\mathcal{B}^{\pi}_1, \mathcal{B}^{\pi}_2, \ldots, \mathcal{B}^{\pi}_K)$ be its block partition. Apply Johnson's rule within each block $\mathcal{B}^{\pi}_k$ $(k = 1, 2, \ldots, K)$ to obtain a permutation π_k, and concatenate

$\pi_1, \pi_2, \ldots, \pi_K$ to form a permutation schedule π'. By Lemma 4.1, $C^w_{\max}(\pi') \le C^w_{\max}(\pi)$. Since π is optimal, π' is optimal as well.

If π' is regular, the lemma follows. Otherwise, repeat the above procedure with π' in place of π. For termination, suppose the first i blocks of the current schedule are already regular (initially $i = 0$). By definition, applying Johnson's rule to a regular block leaves its job sequence unchanged. Hence, the first i blocks remain unchanged. The $(i+1)$-th block in the resulting schedule may contain a different job set, but it satisfies Johnson's rule. Thus the first $i+1$ blocks are regular, so every application increases the number of leading regular blocks by at least one. Since there are at most K nonempty blocks in each schedule, the procedure terminates after at most K iterations and yields an optimal regular schedule. □

Let π be a regular permutation schedule for $F2 \mid\mid C^w_{\max}$ with block partition $\mathcal{B}(\pi) = (\mathcal{B}^\pi_1, \mathcal{B}^\pi_2, \ldots, \mathcal{B}^\pi_K)$. For each $k = 1, 2, \ldots, K$, let π_k denote the schedule obtained by processing the jobs in $\mathcal{B}^\pi_k$ consecutively from time zero in accordance with Johnson's rule. Note that π coincides with the schedule obtained by concatenating $\pi_1, \pi_2, \ldots, \pi_K$. This representation allows us to compute $C_{\max}(\pi)$ and $C^w_{\max}(\pi)$ from parameters defined for each π_k. Specifically, for each k, let $A_k = A(\pi_k)$ and $B_k = B(\pi_k)$ denote the total processing times of the jobs in π_k on M_1 and M_2, respectively, and let $C_k = C(\pi_k)$ denote the makespan of π_k. Given these parameters, we describe an procedure for computing $C_{\max}(\pi)$ and $C^w_{\max}(\pi)$. The procedure starts with π_1 and successively appends $\pi_2, \pi_3, \ldots, \pi_K$ in that order. For $k = 1, 2, \ldots, K$, let $\pi[1:k]$ denote the schedule obtained by concatenating $\pi_1, \pi_2, \ldots, \pi_k$; thus $\pi = \pi[1:K]$. The makespan and weighted makespan of $\pi[1:k]$ are computed recursively. For convenience, set $C_{\max}(\pi[1:0]) = C^w_{\max}(\pi[1:0]) = 0$. For $k = 1, 2, \ldots, K$, by direct calculation and induction on k, we have

$$\begin{aligned} C_{\max}(\pi[1:k]) &= \max\Big\{\sum_{i=1}^{k-1} A_i + C_k, C_{\max}(\pi[1:k-1]) + B_k\Big\} \\ &= \max_{0\le h\le k-1}\Big\{\sum_{i=1}^{h} A_i + C_{h+1} + \sum_{i=h+2}^{k} B_i\Big\} \end{aligned} \tag{30}$$

and

$$\begin{aligned} C^w_{\max}(\pi[1:k]) &= \max\{C^w_{\max}(\pi[1:k-1]), W_k C_{\max}(\pi[1:k])\} \\ &= \max_{1\le h\le k}\{W_h C_{\max}(\pi[1:h])\}. \end{aligned} \tag{31}$$

Hence, $C^w_{\max}(\pi) = C^w_{\max}(\pi[1:K])$.

4.2 An exact DP algorithm

Recall that $W_1 > W_2 > \cdots W_K$. For each job J_j $(j = 1, 2, \ldots, n)$, let $g_j \in \{1, 2, \ldots, K\}$ be the index such that $w_j = W_{g_j}$, i.e., $J_j \in \mathcal{G}_{g_j}$. Assume that the jobs in $\mathcal{J}$ are indexed in accordance with Johnson's rule.

Building on the preceding discussion in Section 4.1, we develop an exact DP algorithm for $F2 \mid\mid C^w_{\max}$. It generates a sequence of sets $\mathcal{H}_0, \mathcal{H}_1, \ldots, \mathcal{H}_n$. Each $\mathcal{H}_j$ consists of K-tuples $(X_1, X_2, \ldots, X_K)$, where $X_k = (A_k, B_k, C_k, L_k)$ for $k = 1, 2, \ldots, K$. A tuple in $\mathcal{H}_j$ $(j = 1, 2, \ldots, n)$ encodes an assignment of jobs $J_1, J_2, \ldots, J_j$ to K schedules (i.e., blocks) $\pi_1(j), \pi_2(j), \ldots, \pi_K(j)$, with jobs in each schedule processed from time zero in increasing order of their indices. For each schedule $\pi_k(j)$, A_k and B_k are the total processing times on M_1 and M_2, respectively, and C_k and L_k are the makespan of $\pi_k(j)$ and the weight of its last job.

Initially, $\mathcal{H}_0$ consists of a single tuple, all of whose $4K$ entries are zero, i.e., $\mathcal{H}_0 = \{\big(\underbrace{(0,0,0,0), \ldots, (0,0,0,0)}_{K \text{ copies}}\big)\}$. In the j-th iteration $(j = 1, 2, \ldots, n)$, $\mathcal{H}_j$ is constructed from $\mathcal{H}_{j-1}$. For each tuple $(X_1, X_2, \ldots, X_K) \in \mathcal{H}_{j-1}$, with $X_k = (A_k, B_k, C_k, L_k)$ for $k = 1, 2, \ldots, K$, the definition of g_j and Lemma 4.2 together imply that this tuple generates g_j new tuples in $\mathcal{H}_j$. If $k \leq g_j$, appending J_j to $\pi_k(j-1)$ yields the new tuple $(X_1, \ldots, X_{k-1}, X'_k, X_{k+1}, \ldots, X_K)$ into $\mathcal{H}_j$, where $X'_k = (A'_k, B'_k, C'_k, L'_k)$ is given by

$$A'_k = A_k + a_j, \;\; B'_k = B_k + b_j, \;\; C'_k = \max\{C_k, A_k + a_j\} + b_j, \;\; L'_k = w_j. \tag{32}$$

The following lemma will be used to remove dominated tuples during the tuple generation, which can be easily verified.

Lemma 4.3. *Let $X = (X_1, X_2, \ldots, X_K)$ and $X' = (X'_1, X'_2, \ldots, X'_K)$ be two tuples in $\mathcal{H}_j$, with $X_k = (A_k, B_k, C_k, L_k)$ and $X'_k = (A'_k, B'_k, C'_k, L'_k)$ for $k = 1, 2, \ldots, K$. If*

$$A_k = A'_k, \;\; B_k = B'_k, \;\; C_k = C'_k, \;\; L_k = L'_k \;\; (k = 1, 2, \ldots, K-1),$$

$L_K = L'_K$, and $C_K \leq C'_K$, then X' is dominated by X and can therefore be deleted from $\mathcal{H}_j$.

By Lemma 4.2, it suffices to consider regular permutation schedules. Therefore, once $\mathcal{H}_n$ has been generated, we discard every tuple $(X_1, X_2, \ldots, X_K)$ with $L_k \notin \{0, W_k\}$ for some k, since concatenating the schedules corresponding to $X_1, X_2, \ldots, X_K$ cannot yield a regular schedule. This does not affect our search for the optimal solution. Then, for each retained tuple in $\mathcal{H}_n$, we compute the makespan and weighted makespan according to (30) and (31), respectively.

Summarizing the above analysis and utilizing Lemma 4.2 and Lemma 4.3, the following algorithm is developed to solve $F2 \mid\mid C^w_{\max}$.

Algorithm 2: An exact DP algorithm for $F2 \mid\mid C^w_{\max}$

Input: Job set $\mathcal{J}$; job parameters a_j, b_j, w_j, and g_j for $j = 1, 2, \ldots, n$, K distinct weights with $W_1 > W_2 > \cdots > W_K$.

Step 1. [Preprocessing] Renumber jobs in $\mathcal{J}$ according to Johnson' rule (Algorithm 1).

Step 2. [Initialization] Set $\mathcal{H}_0 = \{\big(\underbrace{(0,0,0,0),\ldots,(0,0,0,0)}_{K \text{ copies}}\big)\}$, and $\mathcal{H}_j = \emptyset$ for $j = 1, 2, \ldots, n$.

Step 3. [Generation of sets $\mathcal{H}_1, \mathcal{H}_2, \ldots, \mathcal{H}_n$]

for $j = 1$ **to** n **do**

  for *each* $(X_1, X_2, \ldots, X_K) \in \mathcal{H}_{j-1}$ *with* $X_k = (A_k, B_k, C_k, L_k)$, $k = 1, 2, \ldots, K$ **do**

    for $k = 1$ **to** g_j **do**

      $A'_k = A_k + a_j$, $B'_k = B_k + b_j$, $C'_k = \max\{C_{k-1}, A_{k-1} + a_j\} + b_j$, $L'_k = w_j$.

      $X'_k = (A'_k, B'_k, C'_k, L'_k)$.

      Add the tuple $(X_1, \ldots, X_{k-1}, X'_k, X_{k+1}, \ldots, X_K)$ to $\mathcal{H}_j$.

    end

  end

  [Elimination] Among all tuples in $\mathcal{H}_j$ with the same $X_1, X_2, \ldots, X_{K-1}$ and the same L_K, retain only the tuple with the smallest C_K (by Lemma 4.3).

end

Step 4. [Computation of objective values for the tuples in $\mathcal{H}_n$]

for *each* $X = (X_1, X_2, \ldots, X_K) \in \mathcal{H}_n$ *with* $X_k = (A_k, B_k, C_k, L_k)$, $k = 1, 2, \ldots, K$ **do**

  if $L_k \in \{0, W_k\}$ *for each* $k = 1, 2, \ldots, K$ **then**

    Set $T^1_0 = 0$, $T^2_0 = 0$, $Z_0 = 0$.

    for $k = 1$ **to** K **do**

      $T^1_k = T^1_{k-1} + A_k$, $T^2_k = \max\{T^1_{k-1} + C_k,\ T^2_{k-1} + B_k\}$.

      $Z_k = \max\{Z_{k-1}, L_k \cdot T^2_k\}$.

    end

    $C^w_{\max}(X) = Z_K$.

  else

    $C^w_{\max}(X) = +\infty$.

  end

end

Step 5. [Result] Compute $C^w_{\max}(X_{\text{opt}}) = \min\{C^w_{\max}(X) : X \in \mathcal{H}_n\}$ and recover the corresponding optimal schedule π_{opt} from X_{opt} by backtracking.

Output: The optimal schedule π_{opt}.

Theorem 4.1. *Algorithm 2 solves* $F2 \mid\mid C^w_{\max}$ *in* $O(nK(K+1)!2^{K-1}P^{3K-3})$ *time, where* $P = \max\{\sum_{j=1}^n a_j, \sum_{j=1}^n b_j\}$. *For fixed* K, *it runs in* $O(nP^{3K-3})$ *time and is therefore pseudo-polynomial.*

Proof. The correctness follows from three facts: (i) the algorithm enumerates all tuples corresponding to regular schedules, as characterized by Definition 4.2 and Lemma 4.2; (ii)

the elimination operation deletes only dominated tuples, as guaranteed by Lemma 4.3; and (iii) the weighted makespan of a regular schedule is computed using (31).

Step 1 sorts in $O(n \log n)$ time, and Step 2 initializes in $O(n)$ time. For each $j = 1, 2, \ldots, n$ and each tuple $(X_1, X_2, \ldots, X_K) \in \mathcal{H}_j$, write $X_k = (A_k, B_k, C_k, L_k)$, $k = 1, 2, \ldots, K$. The components satisfy $A_k \le \sum_{i=1}^{j} a_i \le P$, $B_k \le \sum_{i=1}^{j} b_i \le P$, $C_k \le \sum_{i=1}^{j} a_i + \sum_{i=1}^{j} b_i \le 2P$, $L_k \in \{0, W_k, W_{k+1}, \ldots, W_K\}$ for $k = 1, 2, \ldots, K$. Because $\sum_{k=1}^{K} A_k = \sum_{i=1}^{j} a_i$ and $\sum_{k=1}^{K} B_k = \sum_{i=1}^{j} b_i$, only $K-1$ of the A_k and $K-1$ of the B_k are independent. After elimination, the number of distinct tuples in $\mathcal{H}_j$ is at most $O((K+1)!P^{2K-2}(2P)^{K-1}) = O((K+1)!2^{K-1}P^{3K-3})$. In addition, each tuple in $\mathcal{H}_{j-1}$ generates $g_j \le K$ new tuples in $\mathcal{H}_j$. Thus, $\mathcal{H}_j$ can be generated in $O(K(K+1)!2^{K-1}P^{3K-3})$ time. Summing the cost of Step 3 over its n iterations gives $O(nK(K+1)!2^{K-1}P^{3K-3})$ time. Step 4 computes the objective value of each tuple in $\mathcal{H}_n$ in $O(K)$ time and thus requires $O(K(K+1)!2^{K-1}P^{3K-3})$ time. Step 5 requires $O((K+1)!2^{K-1}P^{3K-3})$ time. Consequently, the algorithm runs in $O(nK(K+1)!2^{K-1}P^{3K-3})$ time. □

Applying Algorithm 2 to Example 4.1, we obtain an optimal schedule $\pi^* = (3, 5, 6, 2, 1, 4)$, with blocks $\mathcal{B}_1^{\pi^*} = \{J_3\}$, $\mathcal{B}_2^{\pi^*} = \{J_5, J_6, J_2\}$, and $\mathcal{B}_3^{\pi^*} = \{J_1, J_4\}$ (see Figure 5). The weighted makespan of π^* is $C_{\max}^{w}(\pi^*) = \max\{5 \times 15, 3 \times 24, 2 \times 33\} = 75$.

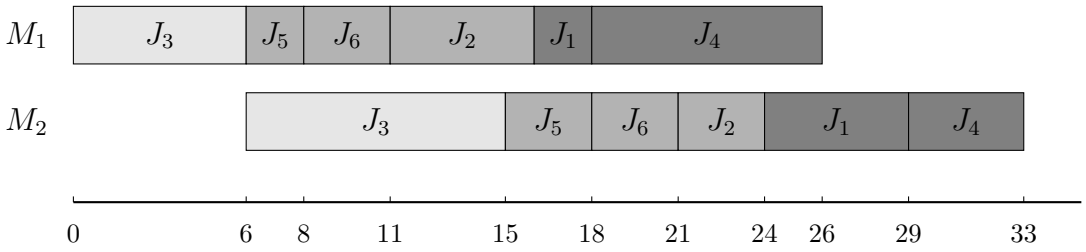


Figure 5: An optimal scheule π^*

4.3 An FPTAS for fixed number of weights

Let $0 < \epsilon \le 1$ be real, and let $\delta = 1 + \epsilon/(2n)$. Define the function $\Delta(\cdot)$ on $[0, +\infty)$ by

$$\Delta(z) = \lfloor \log_\delta z \rfloor \quad (z \ge 1), \qquad \Delta(0) = -1. \tag{33}$$

Equivalently, if k is an integer that satisfies $\delta^k \le z < \delta^{k+1}$, then $\Delta(z) = k$.

For fixed K, we apply the geometric rounding technique to transform Algorithm 2 into an FPTAS (which we call Algorithm 3). In contrast to Algorithm 2, the proposed algorithm may remove some non-dominated states in the tuple set generation procedure. Let $\mathcal{H}_j^{\epsilon}$ be the set of tuples generated by Algorithm 3 in the j-th iteration. For each tuple $(X_1, X_2, \ldots, X_K) \in \mathcal{H}_j^{\epsilon}$, define its label to be $(Y_1, Y_2, \ldots, Y_K)$, where $X_k = (A_k, B_k, C_k, L_k)$ for $k = 1, 2, \ldots, K$; the first $K-1$ labels are

$$Y_k = (\Delta(A_k), \Delta(B_k), \Delta(C_k), L_k), \quad 1 \le k \le K-1,$$

while the last label is

$$Y_K = (\Delta(A_K), \Delta(B_K), L_K).$$

By construction, for each $1 \le k \le K$ with $A_k \ge 1$, the rounded value $\delta^{\Delta(A_k)}$ approximates A_k up to a small multiplicative error; likewise, $\delta^{\Delta(B_k)}$ and $\delta^{\Delta(C_k)}$ approximate B_k and C_k, respectively. Hence, whenever two tuples share a label, one of them is eliminated during tuple elimination procedure. A formal description of Algorithm 3 is given below.

Algorithm 3: An FPTAS for $F2 \,||\, C^w_{\max}$

Input: Job set $\mathcal{J}$; error parameter ϵ; job parameters a_j, b_j, w_j, and g_j for $j = 1, 2, \ldots, n$; K distinct weights with $W_1 > W_2 > \cdots > W_K$.

Step 1. [Preprocessing] Renumber jobs in $\mathcal{J}$ according to Johnson' rule (Algorithm 1).

Step 2. [Initialization] Set $\delta = 1 + \epsilon/(2n)$. Let $S_0 = \Big(\underbrace{(0,0,0,0),\ldots,(0,0,0,0)}_{K \text{ copies}}\Big)$, and let $R_0 = \Big(\underbrace{(-1,-1,-1,0),\ldots,(-1,-1,-1,0)}_{K-1 \text{ copies}},(-1,-1,0)\Big)$ be its label. Initialize $\mathcal{H}^\epsilon_0 = \{S_0\}$ and $\mathcal{H}^\epsilon_j = \emptyset$ for $j = 1, 2, \ldots, n$.

Step 3. [Generation of sets $\mathcal{H}^\epsilon_1, \mathcal{H}^\epsilon_2, \ldots, \mathcal{H}^\epsilon_n$]

for $j = 1$ **to** n **do**

  for *each* $(X_1, X_2, \ldots, X_K) \in \mathcal{H}^\epsilon_{j-1}$ *with* $X_k = (A_k, B_k, C_k, L_k)$, $k = 1, 2, \ldots, K$ **do**

    for $k = 1$ **to** g_j **do**

      $A'_k = A_k + a_j$, $B'_k = B_k + b_j$, $C'_k = \max\{C_{k-1}, A_{k-1} + a_j\} + b_j$, $L'_k = w_j$.

      $X'_k = (A'_k, B'_k, C'_k, L'_k)$.

      if $k \le K - 1$ **then**

        $Y'_k = (\Delta(A'_k), \Delta(B'_k), \Delta(C'_k), L'_k)$.

      else

        $Y'_K = (\Delta(A'_K), \Delta(B'_K), L'_K)$.

      end

      Add the tuple $(X_1, \ldots, X_{k-1}, X'_k, X_{k+1}, \ldots, X_K)$ to $\mathcal{H}^\epsilon_j$ and attach the associated label $(Y_1, \ldots, Y_{k-1}, Y'_k, Y_{k+1}, \ldots, Y_K)$ to it.

    end

  end

  [Elimination] Among tuples in $\mathcal{H}^\epsilon_j$ sharing identical labels, retain only the one with the smallest C_K.

end

Step 4. [Computation of objective values for the tuples in $\mathcal{H}^\epsilon_n$]

for *each* $X = (X_1, X_2, \ldots, X_K) \in \mathcal{H}^\epsilon_n$ *with* $X_k = (A_k, B_k, C_k, L_k)$, $k = 1, 2, \ldots, K$ **do**

  Perform the same operations as in Step 4 of Algorithm 2.

end

Step 5. [Result] Compute $C^w_{\max}(X') = \min\{C^w_{\max}(X) : X \in \mathcal{H}^\epsilon_n\}$ and recover the corresponding approximate schedule π' from X' by backtracking.

Output: The approximate schedule π'.

In Algorithm 3, the box-elimination operation considers coordinate values that increase

geometrically. Multiplicative errors can occur if, during tuple elimination, each label is represented by the smallest C_X coordinates inside its box rather than by its exact value. Nevertheless, the lemma below demonstrates that the tuples left after elimination form a good approximation to those produced by Algorithm 2.

Lemma 4.4. *Given any tuple $(X_1, X_2, \ldots, X_K) \in \mathcal{H}_j$ generated by Algorithm 2, where $X_k = (A_k, B_k, C_k, L_k)$ for $k = 1, 2, \ldots, K$, Algorithm 3 returns a tuple $(\widetilde{X_1}, \widetilde{X_2}, \ldots, \widetilde{X_K}) \in \mathcal{H}_j^\epsilon$, with $\widetilde{X_k} = (\widetilde{A_k}, \widetilde{B_k}, \widetilde{C_k}, L_k)$, such that*

$$\widetilde{A_k} \le \delta^j A_k, \quad \widetilde{B_k} \le \delta^j B_k, \quad \widetilde{C_k} \le \delta^j C_k, \quad k = 1, 2, \ldots, K. \tag{34}$$

Proof. We prove the result by induction on $j = 1, 2, \ldots, n$. For the base case $j = 1$, J_1 can be assigned to each of the g_1 schedules (each induced by a block) in Algorithm 2 and Algorithm 3. Since these g_1 tuples receive distinct labels in Algorithm 3, it follows that $\mathcal{H}_1^\epsilon = \mathcal{H}_1$, so (34) holds for $j = 1$.

Assume that (34) holds for $j - 1$ (the induction hypothesis), where $j \ge 2$. Let $(X_1, X_2, \ldots, X_K) \in \mathcal{H}_j$ be a tuple obtained from Algorithm 2 for the first j jobs, where $X_k = (A_k, B_k, C_k, L_k)$ for $k = 1, 2, \ldots, K$. When implementing Algorithm 2, the tuple $(X_1, X_2, \ldots, X_K)$ is constructed from a tuple $(X'_1, X'_2, \ldots, X'_K) \in \mathcal{H}_{j-1}$, where $X'_k = (A'_k, B'_k, C'_k, L'_k)$ for $k = 1, 2, \ldots, K$. Specifically, J_j is assigned to one of the g_j schedules corresponding to $(X'_1, X'_2, \ldots, X'_K)$ for the first $j-1$ jobs. Based on the induction hypothesis, there exists a tuple $(\widetilde{X'_1}, \widetilde{X'_2}, \ldots, \widetilde{X'_K}) \in \mathcal{H}_{j-1}^\epsilon$, where $\widetilde{X'_k} = (\widetilde{A'_k}, \widetilde{B'_k}, \widetilde{C'_k}, L'_k)$, such that

$$\widetilde{A'_k} \le \delta^{j-1} A'_k, \quad \widetilde{B'_k} \le \delta^{j-1} B'_k, \quad \widetilde{C'_k} \le \delta^{j-1} C'_k, \quad k = 1, 2, \ldots, K. \tag{35}$$

To prove that (34) holds for j, consider the g_j possible cases in which J_j is assigned to one of these schedules.

Let J_j be assigned to the h-th schedule in Algorithm 2 for some $h \in \{1, 2, \ldots, g_j\}$. Then

$$\begin{aligned} &A_k = A'_k, \quad B_k = B'_k \ , C_k = C'_k \ , L_k = L'_k, \quad k \in \{1, 2, \ldots, K\} \setminus \{h\}; \\ &A_h = A'_h + a_j, \quad B_h = B'_h + b_j, \quad C_h = \max\{C'_h, A'_h + a_j\} + b_j, \quad L_h = w_j. \end{aligned} \tag{36}$$

Since $(\widetilde{X'_1}, \widetilde{X'_2}, \ldots, \widetilde{X'_K}) \in \mathcal{H}_{j-1}^\epsilon$, the tuple $(\widetilde{X'_1}, \widetilde{X'_2}, \ldots, \widetilde{X'_{h-1}}, \overline{X'_h}, \widetilde{X'_{h+1}}, \ldots, \widetilde{X'_K})$ is added to $\mathcal{H}_j^\epsilon$ in Algorithm 3, where $\overline{X'_h} = (\overline{A'_h}, \overline{B'_h}, \overline{C'_h}, \overline{L'_h})$ is defined by

$$\overline{A'_h} = \widetilde{A'_h} + a_j, \quad \overline{B'_h} = \widetilde{B'_h} + b_j, \quad \overline{C'_h} = \max\{\widetilde{C'_h}, \widetilde{A'_h} + a_j\} + b_j, \quad \overline{L'_h} = w_j. \tag{37}$$

Because two tuples can share a label, $(\widetilde{X'_1}, \widetilde{X'_2}, \ldots, \widetilde{X'_{h-1}}, \overline{X'_h}, \widetilde{X'_{h+1}}, \ldots, \widetilde{X'_K})$ may be discarded in Algorithm 3 in favor of another tuple $(\widetilde{X_1}, \widetilde{X_2}, \ldots, \widetilde{X_K}) \in \mathcal{H}_j^\epsilon$, where $\widetilde{X_k} = (\widetilde{A_k}, \widetilde{B_k}, \widetilde{C_k}, \widetilde{L_k})$ for $k = 1, 2, \ldots, K$. For this tuple, by (35)–(37) and the construction in Algorithm 3, we have

$$\begin{cases} \widetilde{A_k} \le \delta \widetilde{A'_k} \le \delta^j A'_k = \delta^j A_k \\ \widetilde{B_k} \le \delta \widetilde{B'_k} \le \delta^j B'_k = \delta^j B_k \\ \widetilde{L_k} = \overline{L'_k} = L'_k = L_k \end{cases} \quad \text{if} \quad k \in \{1, 2, \ldots, K\} \setminus \{h\}; \tag{38}$$

$$\begin{cases} \widetilde{A_h} \le \delta\overline{A'_h} = \delta(\widetilde{A'_h} + a_j) \le \delta(\delta^{j-1}A'_h + a_j) \le \delta^j(A'_h + a_j) = \delta^j A_h \\ \widetilde{B_h} \le \delta\overline{B'_h} = \delta(\widetilde{B'_h} + b_j) \le \delta(\delta^{j-1}B'_h + b_j) \le \delta^j(B'_h + b_j) = \delta^j B_h \\ \widetilde{L_h} = \overline{L'_h} = w_j = L_h \end{cases}; \tag{39}$$

$$\begin{cases} \widetilde{C_k} \le \delta\widetilde{C'_k} \le \delta^j C'_k = \delta^j C_k & \text{if } \ k \in \{1,2,\ldots,K-1\} \setminus \{h\} \\ \widetilde{C_K} \le \widetilde{C'_K} \le \delta^{j-1}C'_K \le \delta^j C_K & \text{if } \ K \ne h \end{cases}; \tag{40}$$

$$\begin{cases} \widetilde{C_h} \le \delta\overline{C'_h} = \delta(\max\{\widetilde{C'_h}, \widetilde{A'_h} + a_j\} + b_j) \\ \qquad \le \delta(\max\{\delta^{j-1}C'_h, \delta^{j-1}A'_h + a_j\} + b_j) & \text{if } \ h \ne K \\ \qquad \le \delta^j(\max\{C'_h, A'_h + a_j\} + b_j) = \delta^j C_h \\ \widetilde{C_K} \le \overline{C'_K} = \max\{\widetilde{C'_K}\widetilde{A'_K} + a_j\} + b_j \\ \qquad \le \max\{\delta^{j-1}C'_K, \delta^{j-1}A'_K + a_j\} + b_j & \text{if } \ h = K \\ \qquad \le \delta^{j-1}(\max\{C'_K, A'_K + a_j\} + b_j) \le \delta^j C_K \end{cases}. \tag{41}$$

By (38)-(41), (34) holds for j when J_j is assigned to the h-th schedule. Since h is arbitrary, this covers every possible assignment of J_j. Thus, the induction step is established for j. □

To analyze the approximation error and time complexity of Algorithm 3, we use the following two well-known inequalities. The first follows because $f(x) = x\ln x - (x-1)$ is increasing on $[1, +\infty)$ and $f(1) = 0$. The second follows because $g(x) = (1 + \frac{x}{n})^n - (1 + 2x)$ is convex on $[0, 1]$, with $g(0) = 0$ and $g(1) \le 0$.

Lemma 4.5. *For any real $x \ge 1$, $\ln x \ge (x-1)/x$.*

Lemma 4.6. *For any real $0 < x \le 1$, $(1 + x/n)^n \le 1 + 2x$.*

Theorem 4.2. *For fixed K, Algorithm 3 is an FPTAS for $F2 \ || \ C^w_{\max}$ with $O((n^{3K}/\epsilon^{3K-1})(\log P)^{3K-1})$ running time.*

Proof. We first bound the approximation error of the algorithm. Let π' be the approximate schedule returned by algorithm 3, and let π_{opt} be the optimal schedule returned by Algorithm 2. Denote by $(X^*_1, X^*_2, \ldots, X^*_K) \in \mathcal{H}_n$ the tuple corresponding to π_{opt}, where $X^*_k = (A^*_k, B^*_k, C^*_k, L^*_k)$ for $k = 1, 2, \ldots, K$. By (30) and (31), we have

$$C^w_{\max}(\pi_{\text{opt}}) = \max_{1\le k\le K}\Big\{W_k \max_{0\le h\le k-1}\Big(\sum_{i=1}^{h} A^*_i + C^*_{h+1} + \sum_{i=h+2}^{k} B^*_i\Big)\Big\}. \tag{42}$$

By Lemma 4.4, there exists a tuple $(\widetilde{X^*_1}, \widetilde{X^*_2}, \ldots, \widetilde{X^*_K}) \in \mathcal{H}^\epsilon_n$ generated by Algorithm 3, with $\widetilde{X^*_k} = (\widetilde{A^*_k}, \widetilde{B^*_k}, \widetilde{C^*_k}, L^*_k)$, such that

$$\widetilde{A^*_k} \le \delta^n A^*_k, \quad \widetilde{B^*_k} \le \delta^n B^*_k, \quad \widetilde{C^*_k} \le \delta^n C^*_k, \quad k = 1, 2, \ldots, K. \tag{43}$$

Let $\widetilde{\pi_{\text{opt}}}$ be the schedule corresponding to $(\widetilde{X_1^*}, \widetilde{X_2^*}, \ldots, \widetilde{X_K^*})$. Again by (30) and (31), we have

$$C_{\max}^w(\widetilde{\pi_{\text{opt}}}) = \max_{1\le k\le K}\Big\{W_k \max_{0\le h\le k-1}\Big(\sum_{i=1}^{h}\widetilde{A_i^*} + \widetilde{C_{h+1}^*} + \sum_{i=h+2}^{k}\widetilde{B_i^*}\Big)\Big\}. \tag{44}$$

Recall that $0 < \epsilon \le 1$ and $\delta = 1 + \epsilon/(2n)$. Then by Lemma 4.6 and (42)–(44), we have

$$\begin{aligned} C_{\max}^w(\widetilde{\pi_{\text{opt}}}) &\le \max_{1\le k\le K}\Big\{W_k \max_{0\le h\le k-1}\Big(\sum_{i=1}^{h}\delta^n A_i^* + \delta^n C_{h+1}^* + \sum_{i=h+2}^{k}\delta^n B_i^*\Big)\Big\} \\ &= \delta^n \max_{1\le k\le K}\Big\{W_k \max_{0\le h\le k-1}\Big(\sum_{i=1}^{h} A_i^* + C_{h+1}^* + \sum_{i=h+2}^{k} B_i^*\Big)\Big\} \\ &\le (1+\epsilon) C_{\max}^w(\pi_{\text{opt}}). \end{aligned}$$

Thus, from the execution of Algorithm 3, it returns an approximate schedule π' with $C_{\max}^w(\pi') \le C_{\max}^w(\widetilde{\pi_{\text{opt}}}) \le (1+\epsilon) C_{\max}^w(\pi_{\text{opt}})$.

We next bound the time complexity of the algorithm. Step 1 sorts in $O(n \log n)$ time, and Step 2 initializes in $O(n)$ time. In Step 3, at most one tuple is retained per label after elimination. For each $(X_1, X_2, \ldots, X_K) \in \mathcal{H}_j^\epsilon$ $(j = 1, 2, \ldots, n)$, the corresponding label is $(Y_1, Y_2, \ldots, Y_K)$, where $X_k = (A_k, B_k, C_k, L_k)$ for $k = 1, 2, \ldots, K$, $Y_k = (\Delta(A_k), \Delta(B_k), \Delta(C_k), L_k)$ for $1 \le k \le K-1$, and $Y_K = (\Delta(A_K), \Delta(B_K), L_K)$. Note that $A_k \le P$, $B_k \le P$, $C_k \le 2P$, and $L_k \in \{0, W_k, W_{k+1}, \ldots, W_K\}$ for $k = 1, 2, \ldots, K$. By Lemma 4.5, the number of distinct values attained by $\Delta(A_k)$ is upper bounded by $1 + \lfloor \log_\delta P \rfloor = 1 + \lfloor \ln P / \ln \delta \rfloor \le 1 + \lfloor (1 + 2n/\epsilon) \ln P \rfloor = O((n/\epsilon) \log P)$. Analogously, the number of distinct values attained by $\Delta(B_k)$ and $\Delta(C_k)$ are also upper bounded by $O((n/\epsilon) \log P)$. Because each tuple yields at most K new tuples, after elimination, the number of retained tuples in $\mathcal{H}_j^\epsilon$ is upper bounded by $O((n^{3K-1}/\epsilon^{3K-1})(\log P)^{3K-1})$. Summing the cost of Step 3 over its n iterations gives $O((n^{3K}/\epsilon^{3K-1}) \log P)$ time. Step 4 computes the objective value of each tuple in $\mathcal{H}_n^\epsilon$ in $O(K)$ time, and thus requires $O((n^{3K-1}/\epsilon^{3K-1})(\log P)^{3K-1})$ time in total. Step 5 has the same time complexity. Consequently, the algorithm runs in $O((n^{3K}/\epsilon^{3K-1})(\log P)^{3K-1})$ time. □

4.4 The bounded max-to-min weight ratio

Consider the bounded max-to-min ratio weight case where $w_{\max}/w_{\min} \le C$ for some constant C. Here, $w_{\max} = \max_{1\le j\le n}\{w_j\}$ is the maximum weight of the n jobs, and $w_{\min} = \min_{1\le j\le n}\{w_j\}$ is the minimum weight. The proof of Theorem 3.1 demonstrates that the problem remains $\mathcal{NP}$-hard even when $C = 2$.

Let $0 < \epsilon \le 1$ be a real number. For $j = 1, 2, \ldots, n$, we round each weight w_j down to the nearest value of the form $w_{\min}(1+\epsilon/2)^k$, i.e., $\overline{w_j} = w_{\min}(1+\epsilon/2)^{\theta_j}$, where θ_j is the unique integer satisfying

$$w_{\min}(1+\epsilon/2)^{\theta_j} \le w_j < w_{\min}(1+\epsilon/2)^{\theta_j+1}. \tag{45}$$

By combining job-weight rounding with the FPTAS in Section 4.3, a PTAS for the bounded max-to-min ratio weight case is given as follows.

Algorithm 4: A PTAS for $F2 \,||\, C^w_{\max}$ with bounded max-to-min weight ratio

Input: Job set $\mathcal{J}$; error parameter ϵ; job parameters a_j, b_j, and w_j for $j = 1, 2, \ldots, n$.

1 **Step 1.** Construct the rounded instance $\mathcal{I}'$ from the original instance $\mathcal{I}$: replace each job J_j by J'_j with $w'_j = w_{\min}(1+\epsilon/2)^{\theta_j}$, where θ_j is the integer given by (45); keep all other job parameters unchanged.

2 **Step 2.** Let $W_1 > W_2 > \cdots > W_K$ be the ordered sequence of distinct rounded weights in $\mathcal{I}'$. Run Algorithm 3 on $\mathcal{I}'$ to obtain a $(1+\epsilon/3)$-approximation schedule π_ϵ.

Output: The approximate schedule π_ϵ.

For a schedule π, let $C^w_{\max}(\pi, \mathcal{I}) = \max_{1\le j\le n}\{w_{\pi(j)}C_{\pi(j)}(\pi, \mathcal{I})\}$ and $C^w_{\max}(\pi, \mathcal{I}') = \max_{1\le j\le n}\{w'_{\pi(j)}C_{\pi(j)}(\pi, \mathcal{I}')\}$ denote its weighted makespan on instances $\mathcal{I}$ and $\overline{\mathcal{I}}$, respectively. Here, $C_{\pi(j)}(\pi, \mathcal{I})$ and $C_{\pi(j)}(\pi, \mathcal{I}')$ are the completion times of $J_{\pi(j)}$ under π on $\mathcal{I}$ and $\mathcal{I}'$, respectively.

Theorem 4.3. *When $w_{\max}/w_{\min} \le C$, Algorithm 4 is a PTAS for $F2 \,||\, C^w_{\max}$ with $O((n^{O(1/\epsilon)}/\epsilon^{O(1/\epsilon)})(\log P)^{O(1/\epsilon)})$ running time.*

Proof. Let π_{opt} be an optimal schedule for $\mathcal{I}$, and let π'_{opt} be an optimal schedule for $\mathcal{I}'$. Because $\mathcal{I}$ and $\mathcal{I}'$ differ only in their job weights, a fixed schedule π yields the same job completion times on both instances, i.e., $C_{\pi(j)}(\pi, \mathcal{I}) = C_{\pi(j)}(\pi, \mathcal{I}')$. Thus, from the fact that $w'_j \le w_j < (1+\epsilon/2)w'_j$, $j = 1, 2, \ldots, n$, we have

$$C^w_{\max}(\pi_\epsilon, \mathcal{I}) < (1+\epsilon/2)C^w_{\max}(\pi_\epsilon, \mathcal{I}') \tag{46}$$

and

$$C^w_{\max}(\pi'_{\text{opt}}, \mathcal{I}') \le C^w_{\max}(\pi_{\text{opt}}, \mathcal{I}') \le C^w_{\max}(\pi_{\text{opt}}, \mathcal{I}). \tag{47}$$

Since π_ϵ is a $(1+\epsilon/3)$-approximation schedule for $\mathcal{I}'$, we have

$$C^w_{\max}(\pi_\epsilon, \mathcal{I}') \le (1+\epsilon/3)C^w_{\max}(\pi'_{\text{opt}}, \mathcal{I}'). \tag{48}$$

Then, (46)–(48) imply that

$$C^w_{\max}(\pi_\epsilon, \mathcal{I}) \le (1+\epsilon/2)(1+\epsilon/3)C^w_{\max}(\pi'_{\text{opt}}, \mathcal{I}') \le (1+\epsilon)C^w_{\max}(\pi_{\text{opt}}, \mathcal{I}).$$

Let $\mathcal{W} = \{\overline{w_j} : 1 \le j \le n\}$ be the set of distinct rounded job weights in $\mathcal{I}'$, and let $K = |\mathcal{W}|$. Since $w_{\max}/w_{\min} \le C$ and C is a constant, we have $K \le 1 + \lfloor \log_{1+\epsilon/2} C \rfloor = O(1/\epsilon)$. The running time is dominated by Step 2. By Theorem 4.2, it requires $O((n^{3K}/\epsilon^{3K-1})(\log P)^{3K-1}) = O((n^{O(1/\epsilon)}/\epsilon^{O(1/\epsilon)})(\log P)^{O(1/\epsilon)})$ time. Consequently, the overall running time is $O((n^{O(1/\epsilon)}/\epsilon^{O(1/\epsilon)})(\log P)^{O(1/\epsilon)})$. □

5 Heuristic algorithms

This section presents two heuristic methods for instances that are too large to be solved efficiently by the exact methods: an adaptation of the NEH heuristic, which constructs a schedule through sequential job insertion, and a genetic algorithm, which searches the space of job permutations.

5.1 The LW algorithm

Sun (2024) proved that the LW-based greedy algorithm is a $(2-1/m)$-approximation for $Pm \mid\mid C^w_{\max}$. For $F2 \mid\mid C^w_{\max}$, we describe the analogous LW-based algorithm and show that its worst-case approximation ratio is 2.

Algorithm 5: LW's rule for $F2 \mid\mid C^w_{\max}$

Input: Job set $\mathcal{J}$; job parameters a_j, b_j, and w_j for $j = 1, 2, \ldots, n$.

1 **Step 1.** Renumber the jobs in nonincreasing order of their weights (i.e., the LW order), so that $w_1 \geq w_2 \geq \cdots \geq w_n$.

2 **Step 2.** Process the jobs in this order on M_1 and then on M_2, yielding the permutation schedule π_{lw}.

Output: The resulting schedule π_{lw}.

Theorem 5.1. *For $F2 \mid\mid C^w_{\max}$, Algorithm 5 has a worst-case approximation ratio of 2, which is tight, and runs in $O(n \log n)$ time.*

Proof. Since $1 \mid\mid C^w_{\max}$ can be solved in $O(n \log n)$ time by the LW rule (Nong et al., 2011), we have

$$C^w_{\max}(\pi_{\mathrm{opt}}) \geq LB = \max\left\{ \max_{1\leq j\leq n}\left\{w_j \sum_{i=1}^{j} a_i\right\}, \max_{1\leq j\leq n}\left\{w_j \sum_{i=1}^{j} b_i\right\}, \max_{1\leq j\leq n}\left\{w_j(a_j+b_j)\right\}\right\}, \quad (49)$$

where the first two terms are machine-based lower bounds and the last is a job-based lower bound.

By construction, Algorithm 5 yields $\pi_{\mathrm{lw}}(j) = j$ for $j = 1, 2, \ldots, n$. Combining (1) with (2), the weighted makespan of π_{lw} can be written as

$$C^w_{\max}(\pi_{\mathrm{lw}}) = \max_{1\leq j\leq n}\left\{w_{\pi_{\mathrm{lw}}(j)} C_{\pi_{\mathrm{lw}}(j)}\right\} = \max_{1\leq j\leq n}\left\{w_j \max_{1\leq i\leq j}\left(\sum_{h=1}^{i} a_h + \sum_{h=i}^{j} b_h\right)\right\}. \quad (50)$$

By (49) and (50), we have

$$\begin{aligned} C^w_{\max}(\pi_{\mathrm{lw}}) &\le \max_{1\le j\le n}\left\{w_j\sum_{h=1}^{j}a_h + w_j\sum_{h=1}^{j}b_h\right\} \\ &\le \max_{1\le j\le n}\left\{w_j\sum_{h=1}^{j}a_h\right\} + \max_{1\le j\le n}\left\{w_j\sum_{h=1}^{j}b_h\right\} \\ &\le 2C^w_{\max}(\pi_{\mathrm{opt}}). \end{aligned} \tag{51}$$

To show that the bound is tight, consider an instance with two jobs: J_1 has $a_1 = 1+\epsilon$, $b_1 = \epsilon$, and $w_1 = 1+\epsilon$, and J_2 has $a_2 = \epsilon$, $b_2 = 1+\epsilon$, and $w_2 = 1$, where $\epsilon > 0$ is arbitrarily small. Algorithm 5 yields the schedule $\pi_{\mathrm{lw}} = (1,2)$ with $C^w_{\max}(\pi_{\mathrm{lw}}) = 2+\epsilon$, while the optimal schedule is $\pi_{\mathrm{opt}} = (2,1)$ with $C^w_{\max}(\pi_{\mathrm{opt}}) = (1+\epsilon)(1+2\epsilon)$. Hence, $C^w_{\max}(\pi_{\mathrm{lw}})/C^w_{\max}(\pi_{\mathrm{opt}}) \to 2$ as $\epsilon \to 0$. □

5.2 The NEH algorithm

The NEH heuristic is a classical insertion-based constructive method for the permutation flow-shop scheduling problem. In the formulation of Nawaz et al. Nawaz et al. (1983), jobs are placed sequentially: at each iteration, an unassigned job is examined at every feasible position in the current partial schedule, and the position with the smallest resulting makespan is retained. This procedure has proven competitive in diverse scheduling settings. In addition, NEH has recently been used in scheduling research as a construction to be refined (Puka et al., 2024), as a rule for assigning jobs in distributed lot-streaming and cascaded flow shops (Shao et al., 2023; Wang et al., 2024b), and as the core of composite heuristics for a constrained distributed group scheduling problem (Wang et al., 2024a). For the $C^w_{\max}$ objective, two properties are relevant. First, jobs with large total processing time a_j+b_j tend to lengthen the schedule and should be scheduled early to avoid delaying other jobs. Second, for a fixed makespan, jobs with larger weight w_j benefit more from earlier positions, since this reduces the objective value. Accordingly, we prioritize jobs by nonincreasing a_j+b_j, breaking ties by nonincreasing w_j, and insert each job into the position yielding the smallest objective value. The detailed procedure is presented in Algorithm 6. At iteration k, evaluating all k insertion positions requires $O(k^2)$ time, so the overall running time is $O(n^3)$.

Algorithm 6: NEH heuristic for $F2 \mid\mid C^w_{\max}$

Input: Job set $\mathcal{J}$; job parameters a_j, b_j, and w_j for $j = 1, 2, \ldots, n$.

Step 1. [Preprocessing] Renumber the jobs in $\mathcal{J}$ by nonincreasing $a_j + b_j$, and the ties are broken by nonincreasing w_j.

Step 2. [Initialization] $\pi \leftarrow (J_1)$.

Step 3. [Insertion] **for** $k = 2$ **to** n **do**

  $\pi \leftarrow$ insert J_k into π at a position that minimizes $C^w_{\max}$.

end

Output: The resulting schedule π.

5.3 The genetic algorithm

Genetic algorithms (GAs) are commonly applied in scheduling, where operators are tailored to problem structure. Recent flow-shop studies have developed genetic operators for blocking constraints (de Athayde Prata and Fuchigami, 2024), sequence-independent setup times and non-interruption requirements (Qiao et al., 2022), and energy consumption under variable machine speeds (Wang et al., 2023). For the $C^w_{\max}$ objective, an optimal schedule consists of weight-based blocks, and jobs within each block follow Johnson's order (Lemma 4.1). Using this structural property, we design a weight-block genetic algorithm (which we call Algorithm 7) with a weight-class-based mutation operator as its core component.

Each chromosome encodes a job permutation. The initial population contains the schedule constructed from Johnson's rule and $N - 1$ adjacent-swap variants, where N is the population size. Johnson's rule ignores job weights, but it provides a structured, makespan-minimizing schedule as a starting point for the weight-aware search. Parents are selected by fitness-based binary tournament. Partially mapped crossover is applied with probability p_c and mutation with probability p_m. Elitist selection keeps the N best individuals from the combined parent and offspring pools, preserving the best solution across generations.

The distinctive component of Algorithm 7 is its weight-class mutation operator. Rather than moving individual jobs, it removes a whole weight class, keeps the relative order of its jobs, and reinserts the class as a block at the position yielding the smallest $C^w_{\max}$. The class is chosen with equal probability either from classes containing a job that attains the current $C^w_{\max}$ or uniformly at random. Repositioning all jobs of a class at once makes use of the block structure. A forward move can delay the jobs passed over, so the move is not guaranteed to improve the schedule; its retention is left to the subsequent selection. To improve schedule quality, a local search sequentially relocates the k jobs with the largest $w_j C_j$ values to the position minimizing $C^w_{\max}$. It stops when a round produces no improvement in $C^w_{\max}$ or after G rounds.

Algorithm 7: The weight-block genetic algorithm (WBGA) for $F2 \,||\, C^w_{\max}$

Input: Job set $\mathcal{J}$; population size N; generations G; offspring λ; probabilities p_c, p_m; reinsertion count k.

Step 1. [Initialization] Build P_0 from Johnson's rule and $N-1$ adjacent-swap variants; evaluate all individuals.
Step 2. [Evolution] **for** $g = 1$ **to** G **do**
  Selection Form the mating pool M_g from P_{g-1} by binary tournament selection.
  Crossover while $|O_g| < \lambda$ **do**
    Draw two parents from M_g. With probability p_c, apply crossover to obtain two children; otherwise copy the parents. Add the children to O_g one by one, stoping once $|O_g| = \lambda$.
  end
  Mutation For each child, apply block mutation with probability p_m; evaluate all children using (1) and (2).
  Survival $P_g \leftarrow$ the N schedules with the minimum $C^w_{\max}$ in $P_{g-1} \cup O_g$.
end
$\pi^* \leftarrow$ the schedule with the minimum $C^w_{\max}$ in P_G.
Step 3. [Local search] **repeat**
  Let $J_1, J_2, \ldots, J_k$ be the jobs of π^* with the largest $w_j C_j$ value.
  For each J_i and each position r, construct the schedule obtained by removing J_i and reinserting it into position r, and evaluate it using (1) and (2). Let π' denote the best resulting schedule.
  if $C^w_{\max}(\pi') < C^w_{\max}(\pi^*)$ **then**
    $\pi^* \leftarrow \pi'$
  end
until *no reinsertion lowers* $C^w_{\max}(\pi^*)$*, or* G *rounds have been performed.*

Output: The best schedule π^*.

6 Numerical experiments

All algorithms were implemented in Python 3.13 and run on a computer equipped with an Intel Core Ultra 5 230F processor and 16 GB of RAM, running Windows 11. The **MILP** model was solved using Gurobi 13.0 with a single thread. Each method was run once per instance, with a time limit of 600 s. All objective values were evaluated using the same recurrence (1).

6.1 Experimental setup

An instance consists of n jobs, where each job J_j has processing times a_j and b_j on machines M_1 and M_2, respectively, and a weight w_j. The processing times are independently

drawn from $\mathcal{U}(1,40)$ or $\mathcal{U}(1,80)$, while the weights are drawn from $\{1,\dots,K\}$. A fixed random seed is assigned to each configuration to ensure that all methods are tested on identical instances and that the experiments are reproducible.

Three groups of test instances are considered, covering both processing-time ranges. Each configuration contains 20 independently generated instances. Group 1 evaluates the performance of **DP** across different instance sizes, with $n \in \{40,60,80,100\}$, $K=2$, and weights restricted to $\{1,10\}$, resulting in eight configurations. Group 2 compares the **MILP** model with **LW**, **NEH**, and **GA** for $n = 10,15,\dots,60$, resulting in 22 configurations. Group 3 compares the three heuristic methods, **LW**, **NEH**, and **GA**, on larger instances with $n = 100,120,\dots,400$, resulting in 32 configurations. Groups 2 and 3 use $K=30$, with weights drawn from $\{1,2,\dots,30\}$.

For **GA**, the population size is set to $N = \min\{20, \max\{6, \lceil n/2 \rceil\}\}$. The number of generations is set to $G = \max\{2, \lfloor n/4 \rfloor\}$ for small instances and $G = 11 + \lfloor (n-100)/10 \rfloor$ for large instances. These parameter settings are designed to balance solution quality and computational time.

Algorithmic performance is evaluated using the relative percentage gap:

$$\text{gap}_A = \frac{z_A - z_{\text{base}}}{z_{\text{base}}} \times 100\%, \tag{52}$$

where z_A is the objective value obtained by **method A** ($\mathbf{DP}, \mathbf{LW}, \mathbf{NEH}, \mathbf{GA}$), and z_{base} is the reference value. For the small-scale instances, z_{base} is the **MILP** optimum whenever optimality is proven by the solver. For the large-scale instances, it is the lower bound LB of (49), the maximum of the two machine-based bounds and the job-based bound. Thus, large-scale gaps measure the distance from a common lower bound rather than from competing heuristic solutions. This bound is tight only when the heaviest jobs also appear earliest.

6.2 Computational results

The five methods exhibit different trade-offs between solution quality, computational cost and generality, with none dominating across all instances.

6.2.1 The dynamic program

The **DP** is the only exact method considered and guarantees optimality whenever it terminates. It solves all twenty instances in seven of the eight configurations, with mean running times of 1.47 s and 4.98 s for forty-job instances under $[1,40]$ and $[1,80]$, respectively (Table 2). However, its running time increases rapidly with instance size. Under $[1,40]$, the mean running time rises from 1.47 s for forty jobs to 89.59 s for one hundred jobs, a 61-fold increase, while the mean objective value increases by only a factor of 2.5. Expanding the processing-time range to $[1,80]$ further increases the mean running time by factors of 2.9–3.7.

For one hundred-job instances under $[1, 80]$, six of the twenty instances reach the 600 s time limit, and the remaining fourteen require an average of 329.01 s. Moreover, the **DP** cannot handle a third weight class due to memory limitations, restricting its applicability to $K = 2$. These results indicate that the **DP** is effective for relatively small instances with few distinct weights but has limited scalability.

Table 2: The dynamic program, $K = 2$

n	$a_j, b_j \in [1, 40]$					$a_j, b_j \in [1, 80]$				
	$WC_{\max}$		time (ms)		opt.	$WC_{\max}$		time (ms)		opt.
	avg	max	avg	max		avg	max	avg	max	
40	4335.0	5480.0	1.5e3	4.3e3	20/20	8604.5	10590.0	5.0e3	2.4e4	20/20
60	6340.0	7270.0	1.4e4	3.1e4	20/20	12344.5	15020.0	5.3e4	1.1e5	20/20
80	8221.0	9750.0	4.5e4	1.1e5	20/20	17341.5	19580.0	1.3e5	2.8e5	20/20
100	10647.0	12350.0	9.0e4	1.7e5	20/20	21544.3	24240.0	3.3e5	6.0e5	14/20

Averages at $n = 100$ on $[1, 80]$ cover the 14 instances settled within the 600 s limit; the other six are cut short.

6.2.2 The small scale

Tables 3 to 4 report the objective values, running times and gaps of the four methods under $[1, 40]$ and $[1, 80]$, respectively.

The **MILP** model solves 436 of the 440 instances to optimality, providing reference values. Under $[1, 40]$, its mean running time increases from 70 ms for ten jobs to 160.7 s for sixty jobs, with four instances at the largest size reaching the time limit. The wider processing-time range roughly doubles the objective values but has no consistent effect on running time.

NEH outperforms **LW** in all 22 configurations, achieving overall mean gaps of 3.14% and 5.77%, respectively, and worst individual-instance gaps of 24.89% and 21.49%. This improvement comes at a higher computational cost: **NEH** takes 2.66 ms on average, compared with 0.02 ms for **LW**, reflecting its sequential evaluation of insertion positions versus the latter's single-pass construction. Thus, **LW** is preferable when computational efficiency outweighs solution quality.

The **GA** shows the opposite trend, with its mean gap decreasing from 12.61% to 1.70% under $[1, 40]$ and from 14.29% to 3.90% under $[1, 80]$ as the number of jobs increases from ten to sixty. It outperforms both constructive methods at sixty jobs. Nevertheless, its overall mean gap remains the largest (10.07%), and its performance is non-monotonic, with a worst individual-instance gap of 57.53% at twenty-five jobs. This may reflect the limited search budget at smaller sizes, where six individuals and two generations for ten jobs provide insufficient opportunities to establish the block structure. Its mean running time rises from

0.64 ms to 1.69 s, remaining well below that of the **MILP** model. At sixty jobs, however, the mean gaps of the three heuristics differ by less than three percentage points.

Overall, the small-scale results distinguish the methods primarily by computational cost. The **MILP** model alone certifies optimality but becomes increasingly expensive as the instance size grows. Among the heuristics, **NEH** offers the best quality-cost trade-off, **LW** sacrifices some solution quality for substantially lower running times, while **GA** performs worst in solution quality and incurs the highest computational cost (Figure 6). Hence, **NEH** is the preferred heuristic for small instances unless optimality certification is required. The effectiveness of the additional search effort invested in **GA** on larger instances is examined next.

Table 3: The small scale, $a_j, b_j \in [1, 40]$: MILP, LW, NEH, GA

n	MILP				LW				NEH				GA			
	$WC_{\max}$		time (ms)		gap		time (ms)		gap		time (ms)		gap		time (ms)	
	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
10	2607.8	4369.0	7.0e1	1.2e2	7.3	17.2	8.8e−3	2.5e−2	**3.4**	24.9	7.3e−2	7.9e−2	12.6	30.6	6.4e−1	7.2e−1
15	3758.2	5049.0	1.9e2	4.7e2	6.8	16.9	9.9e−3	1.9e−2	**2.1**	8.5	1.9e−1	2.0e−1	18.2	49.5	2.6e0	2.9e0
20	4811.3	5967.0	5.6e2	1.9e3	6.2	19.9	1.3e−2	2.3e−2	**2.7**	11.7	3.8e−1	4.1e−1	12.1	33.0	1.0e1	1.1e1
25	5590.8	7059.0	1.5e3	5.7e3	5.6	21.5	1.4e−2	2.5e−2	**3.7**	17.5	7.0e−1	7.4e−1	15.3	57.5	2.8e1	3.3e1
30	6888.1	8025.0	2.8e3	1.0e4	5.6	14.9	1.4e−2	2.6e−2	**3.9**	10.8	1.1e0	1.2e0	11.6	26.7	6.2e1	7.0e1
35	7719.7	9373.0	8.4e3	3.6e4	5.7	12.2	1.4e−2	2.4e−2	**3.2**	13.8	1.8e0	1.8e0	12.8	38.7	1.2e2	1.3e2
40	8830.8	10309.0	2.1e4	1.2e5	4.9	10.6	1.7e−2	3.0e−2	**2.4**	6.2	2.5e0	2.6e0	7.0	17.5	2.4e2	2.5e2
45	9669.2	11791.0	3.1e4	2.5e5	6.0	19.0	1.8e−2	3.8e−2	**3.7**	10.3	3.5e0	3.7e0	8.8	24.7	4.1e2	4.5e2
50	10904.9	12558.0	8.4e4	5.1e5	4.5	13.0	1.8e−2	3.1e−2	**3.6**	8.4	4.8e0	4.9e0	6.5	22.9	6.7e2	7.3e2
55	11858.3	14340.0	1.1e5	5.2e5	4.7	10.9	2.7e−2	4.7e−2	**3.1**	11.9	6.2e0	6.4e0	4.2	14.6	1.0e3	1.2e3
60	13051.7	15330.0	1.6e5	6.0e5	4.7	11.7	2.5e−2	4.2e−2	4.3	8.8	8.0e0	8.2e0	**1.7**	8.2	1.7e3	1.9e3

Table 4: The small scale, $a_j, b_j \in [1, 80]$: MILP, LW, NEH, GA

n	MILP				LW				NEH				GA			
	$WC_{\max}$		time (ms)		gap		time (ms)		gap		time (ms)		gap		time (ms)	
	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
10	5029.9	7345.0	5.1e1	1.0e2	8.1	17.5	8.5e−3	1.8e−2	**0.8**	5.7	7.2e−2	7.8e−2	14.3	47.5	6.1e−1	6.8e−1
15	7182.1	8624.0	2.9e2	1.8e3	7.7	19.7	9.5e−3	2.0e−2	**2.4**	14.5	1.9e−1	2.1e−1	15.9	37.5	2.6e0	2.9e0
20	9328.9	12103.0	4.5e2	1.8e3	4.5	11.8	1.2e−2	2.9e−2	**2.6**	9.6	3.8e−1	4.1e−1	16.3	39.1	1.0e1	1.2e1
25	10816.3	14100.0	1.6e3	4.6e3	4.7	16.2	1.2e−2	2.1e−2	**2.1**	7.9	7.0e−1	7.4e−1	15.7	37.6	2.8e1	3.0e1
30	12972.5	16005.0	3.2e3	9.5e3	5.6	11.7	1.5e−2	2.9e−2	**2.6**	13.6	1.2e0	1.2e0	14.4	34.2	6.1e1	6.6e1
35	15093.3	17667.0	8.7e3	4.1e4	7.0	18.1	1.8e−2	3.1e−2	**4.3**	7.9	1.8e0	1.8e0	11.5	27.5	1.2e2	1.3e2
40	16757.4	20146.0	2.4e4	1.5e5	6.3	20.8	1.8e−2	3.1e−2	**2.4**	7.6	2.5e0	2.6e0	5.5	15.9	2.5e2	2.7e2
45	19621.4	24528.0	6.8e4	3.8e5	4.8	12.3	1.6e−2	1.9e−2	**2.8**	11.9	3.5e0	3.7e0	5.3	13.1	4.2e2	4.5e2
50	21338.0	25044.0	7.7e4	4.8e5	6.2	12.4	1.9e−2	3.0e−2	**3.8**	11.1	4.7e0	4.8e0	4.2	12.0	6.6e2	7.4e2
55	23019.6	25376.0	1.4e5	4.7e5	5.0	11.4	2.2e−2	3.9e−2	5.0	16.2	6.2e0	6.5e0	**3.5**	14.7	1.1e3	1.2e3
60	26079.5	31320.0	1.9e5	6.0e5	5.0	13.5	2.8e−2	4.4e−2	4.4	15.4	8.0e0	8.2e0	**3.9**	20.8	1.7e3	1.8e3

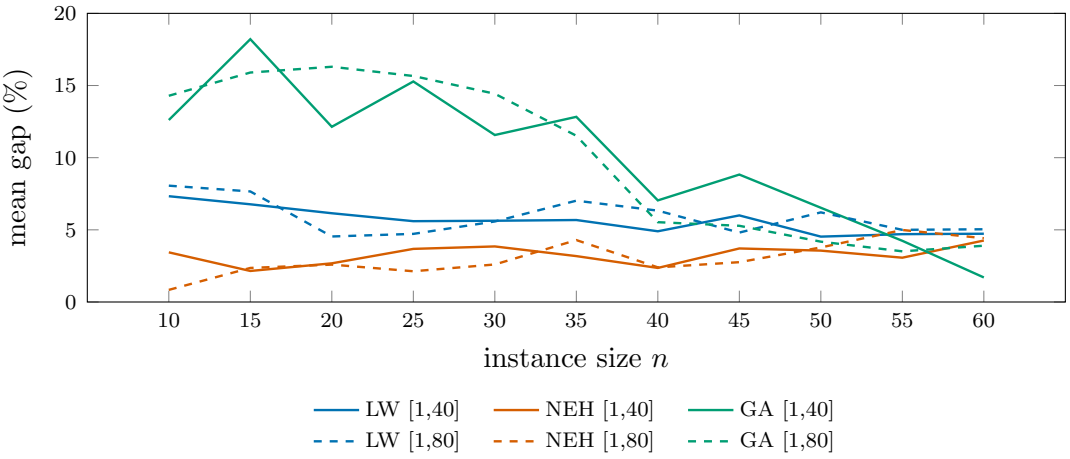


Figure 6: Mean relative gap of the LW and the two heuristics on the small scale, against the optimum the model returned. A solid line is $a_j, b_j \in [1, 40]$, a dashed line $a_j, b_j \in [1, 80]$.

6.2.3 The large scale

Tables 5 to 6 report the gaps and running times of the three heuristics under $[1, 40]$ and $[1, 80]$, respectively.

On the large scale, the ranking varies with instance size. The **LW** achieves the smallest overall mean gap of 2.67% across the 32 configurations, with a worst individual-instance gap of 12.92%, followed by **NEH** at 5.06% and 10.77%, respectively. By contrast, the **GA** improves substantially with instance size: its mean gap decreases from 18.33% to 0.62% under $[1, 40]$ and from 16.01% to 0.82% under $[1, 80]$ between one hundred and four hundred jobs. It overtakes **NEH** at 220 jobs under both ranges and **LW** at 220 and 240 jobs, respectively.

For sufficiently large instances, the **GA** achieves the best solution quality at a higher computational cost. It obtains the best objective values in 328 of the 640 instances, compared with 250 for **LW** and 62 for **NEH**, while its mean running time is 42.2 s, compared with 0.74 s and 0.09 ms, respectively. Unlike the **DP**, it also handles instances with 30 distinct weight classes. The processing-time range has little effect on the overall ranking: the mean gaps of **LW**, **NEH** and **GA** are 2.74%, 4.75% and 4.44% under $[1, 40]$, and 2.59%, 5.36% and 4.96% under $[1, 80]$, respectively. These gaps are measured against the lower bound LB defined in (49), which is tight only when the heaviest jobs appear earliest; they therefore measure deviations from LB rather than true optimality gaps.

Overall, the large-scale results highlight the trade-off between solution quality and computational time. The **LW** is the fastest method and achieves the smallest overall mean gap, while **NEH** offers intermediate performance at negligible computational cost. The **GA** achieves the best solutions on sufficiently large instances but requires substantially more time (Figure 7). The choice therefore depends on whether computational speed or solution

quality is the priority.

Table 5: The large scale, $a_j, b_j \in [1, 40]$: LW, NEH, GA

n	LW				NEH				GA			
	gap		time (ms)		gap		time (ms)		gap		time (ms)	
	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
100	**4.4**	10.0	3.4e−2	6.2e−2	5.2	9.7	3.7e1	6.3e1	18.3	29.2	9.7e2	1.1e3
120	**3.5**	6.5	4.8e−2	1.1e−1	4.8	10.0	6.1e1	6.3e1	13.8	32.1	1.8e3	1.8e3
140	**3.0**	6.6	5.8e−2	1.2e−1	5.3	8.5	9.7e1	1.0e2	9.3	20.6	3.1e3	3.2e3
160	**3.4**	7.5	6.7e−2	1.3e−1	4.2	7.4	1.4e2	1.5e2	6.5	14.4	5.1e3	5.3e3
180	**2.6**	7.8	7.1e−2	9.8e−2	4.9	7.6	2.0e2	2.2e2	5.3	13.8	8.0e3	8.4e3
200	**2.6**	5.6	7.5e−2	1.2e−1	4.4	7.6	2.7e2	2.9e2	4.5	15.7	1.2e4	1.3e4
220	3.4	8.6	8.7e−2	1.1e−1	5.0	8.2	3.6e2	3.8e2	**3.0**	11.7	1.8e4	1.8e4
240	**2.8**	5.9	8.5e−2	1.1e−1	4.7	7.3	4.9e2	5.1e2	2.8	7.4	2.6e4	2.7e4
260	2.6	6.0	1.0e−1	1.5e−1	5.4	8.7	5.7e2	5.8e2	**1.6**	4.7	3.2e4	3.3e4
280	2.5	7.8	9.7e−2	1.2e−1	5.1	7.7	7.2e2	7.4e2	**0.8**	3.9	4.3e4	4.4e4
300	2.0	4.9	1.1e−1	1.8e−1	4.9	8.6	9.0e2	9.1e2	**0.9**	5.0	5.3e4	5.4e4
320	3.0	7.6	1.1e−1	1.2e−1	4.4	7.4	1.1e3	1.1e3	**1.2**	6.3	6.5e4	6.6e4
340	1.5	3.7	1.3e−1	2.4e−1	4.7	7.1	1.3e3	1.3e3	**1.2**	8.0	7.8e4	7.9e4
360	2.2	5.5	1.3e−1	1.6e−1	4.7	7.0	1.6e3	1.6e3	**0.9**	7.1	9.3e4	9.6e4
380	2.3	6.2	1.4e−1	1.5e−1	4.5	8.7	1.9e3	1.9e3	**0.3**	1.5	1.1e5	1.1e5
400	2.0	5.9	1.5e−1	1.7e−1	3.9	8.0	2.2e3	2.2e3	**0.6**	2.7	1.3e5	1.3e5

Table 6: The large scale, $a_j, b_j \in [1, 80]$: LW, NEH, GA

n	LW				NEH				GA			
	gap		time (ms)		gap		time (ms)		gap		time (ms)	
	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
100	**3.6**	8.9	4.1e−2	1.3e−1	4.6	10.8	3.6e1	4.1e1	16.0	30.1	9.6e2	1.0e3
120	**4.4**	12.9	4.3e−2	7.1e−2	5.4	9.3	6.1e1	6.2e1	17.1	38.5	1.8e3	1.8e3
140	**3.5**	10.7	5.2e−2	9.4e−2	4.4	7.7	9.6e1	9.9e1	12.6	25.3	3.1e3	3.2e3
160	**3.2**	11.4	6.8e−2	1.1e−1	5.8	9.9	1.4e2	1.5e2	8.1	25.8	5.1e3	5.2e3
180	**3.0**	7.3	6.7e−2	9.6e−2	5.3	9.0	2.1e2	2.5e2	5.9	14.3	8.3e3	8.5e3
200	**2.7**	8.6	9.3e−2	2.5e−1	4.7	7.2	2.8e2	2.9e2	4.9	10.1	1.2e4	1.3e4
220	**2.9**	8.0	8.0e−2	1.1e−1	4.8	8.7	3.8e2	4.0e2	3.6	9.0	1.8e4	1.9e4
240	2.9	6.4	8.9e−2	1.2e−1	6.0	8.9	4.5e2	4.8e2	**1.9**	6.5	2.4e4	2.5e4
260	1.8	5.3	9.1e−2	1.1e−1	5.2	9.3	5.7e2	5.8e2	**1.6**	8.0	3.2e4	3.3e4
280	2.3	4.8	1.0e−1	1.3e−1	5.5	9.2	7.2e2	7.3e2	**0.8**	4.3	4.3e4	4.4e4
300	2.8	6.3	1.0e−1	1.2e−1	5.4	7.5	9.0e2	9.1e2	**2.0**	11.8	5.3e4	5.4e4
320	1.9	4.7	1.2e−1	2.3e−1	5.7	8.5	1.1e3	1.1e3	**1.4**	6.1	6.5e4	6.6e4
340	1.4	3.0	1.2e−1	1.3e−1	5.6	8.6	1.3e3	1.4e3	**0.8**	3.3	7.8e4	8.0e4
360	1.7	3.2	1.3e−1	1.5e−1	5.8	8.2	1.6e3	1.6e3	**1.1**	7.3	9.3e4	9.4e4
380	1.8	4.0	1.4e−1	1.6e−1	5.9	9.1	1.9e3	1.9e3	**0.8**	2.9	1.1e5	1.1e5
400	1.7	4.6	1.5e−1	2.1e−1	5.8	9.1	2.2e3	2.3e3	**0.8**	2.9	1.3e5	1.3e5

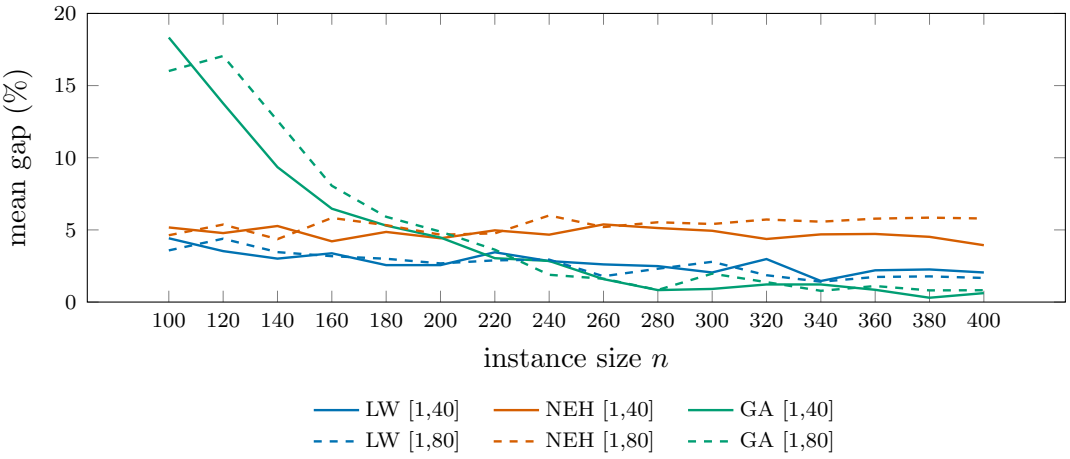


Figure 7: Mean relative gap of the LW and the two heuristics on the large scale, against the lower bound LB. A solid line is $a_j, b_j \in [1, 40]$, a dashed line $a_j, b_j \in [1, 80]$.

7 Conclusion

This paper studies the two-machine flow shop scheduling problem with the maximum weighted completion time objective. The problem is strongly NP-hard, and its optimal schedules exhibit a weight-based block structure, with Johnson's order within each block and blocks arranged in non-increasing order of weight. Exploiting this structure, we develop the **DP**, an FPTAS for a fixed number of weight classes, and three heuristics the **LW** rule, **NEH** and the **GA**.

Several directions for future research remain. First, the problem can be extended to more than two machines. Although the existing **MILP** formulation provides a starting point, the single-permutation property established for two machines does not generally hold for $m > 2$. The extended model must therefore account for machine-specific sequencing decisions and precedence constraints between operations. Investigating the structural properties of optimal schedules in this setting may also provide a basis for developing more efficient exact and approximation algorithms.

Second, for instances with more distinct weight classes, branch-and-bound offers a promising alternative to the **DP**, whose computational requirements increase rapidly with K. The weight-based block structure can guide branching decisions, while the lower bounds can be used to evaluate partial schedules and prune unpromising nodes. Further improvements may be obtained by combining these bounds with dominance rules and heuristic solutions to establish strong upper bounds at early stages of the search.

Third, the proposed heuristics can be extended to distributed flow shops, where job assignment to production shops and job sequencing must be optimized jointly. The **NEH** algorithm can be adapted to evaluate job insertions across candidate shops, accounting for both assignment and sequencing decisions. For the **GA**, the chromosome can be extended to encode shop assignments together with job sequences, while the weight-block mutation can be retained or modified to preserve the structural advantages identified in this study. These extensions may provide effective solution approaches for larger and more complex production environments.

References

Chai, X., Lu, L., Li, W., and Zhang, L. (2018). Best-possible online algorithms for single machine scheduling to minimize the maximum weighted completion time. *Asia-Pacific Journal of Operational Research*, 35(6):1850048.

de Athayde Prata, B. and Fuchigami, H. Y. (2024). A genetic iterated greedy algorithm for the blocking flowshop to minimize total earliness and tardiness. *Journal of Intelligent Manufacturing*, 35(5):2161–2174.

Della Croce, F. and Schau, Q. (2026). Revisiting Johnson's rule for minimizing makespan in the two-machine flow shop scheduling problem. *Operations Research Letters*, 69:107514.

Fan, B., Lu, L., and Shi, Z. (2026). On-line single-machine scheduling to minimize the weighted makespan under (dis)agreeability conditions. *Journal of Industrial and Management Optimization*, 22(8):4138–4150.

Garey, M. R. and Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of $\mathcal{NP}$-Completeness*. W. H. Freeman and Company, San Francisco, CA.

Guo, S., Chen, Y., Mao, Y., and Zhang, X. (2024). Online single-processor scheduling with an unexpected breakdown. *Asia-Pacific Journal of Operational Research*, 41(1):2350005.

Hnaien, F. and Arbaoui, T. (2023). Minimizing the makespan for the two-machine flow shop scheduling problem with random breakdown. *Annals of Operations Research*, 328(2):1437–1460.

Johnson, S. M. (1954). Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68.

Li, W. (2015). A best possible online algorithm for the parallel-machine scheduling to minimize the maximum weighted completion time. *Asia-Pacific Journal of Operational Research*, 32(04):1550030.

Li, W. and Liu, H. (2023). Online NDP-constraint scheduling of jobs with delivery times or weights. *Optimization Letters*, 17(3):591–612.

Liang, X., Lu, L., Sun, X., Yu, X., and Zuo, L. (2024). Online scheduling on a single machine with one restart for all jobs to minimize the weighted makespan. *AIMS Mathematics*, 9(1):2518–2529.

Liu, L. and Urgo, M. (2024a). A branch-and-bound approach to minimise the value-at-risk of the makespan in a stochastic two-machine flow shop. *International Journal of Production Research*, 62(6):2107–2123.

Liu, L. and Urgo, M. (2024b). Robust scheduling in a two-machine re-entrant flow shop to minimise the value-at-risk of the makespan: branch-and-bound and heuristic algorithms based on Markovian activity networks and phase-type distributions. *Annals of Operations Research*, 338(1):741–764.

Lu, L., Zuo, L., Zhang, L., and Ou, J. (2025). Order acceptance and scheduling with weighted makespan. *4OR*, 23(3):247–267.

Mosheiov, G. and Sarig, A. (2023). A note on lot scheduling on a single machine to minimize maximum weighted tardiness. *Journal of Combinatorial Optimization*, 45(5):128.

Nawaz, M., Enscore, E. E., and Ham, I. (1983). A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem. *Omega*, 11(1):91–95.

Nong, Q. Q., Cheng, T. C. E., and Ng, C. T. (2011). Two-agent scheduling to minimize the total cost. *European Journal of Operational Research*, 215(1):39–44.

Pinedo, M. L. (2022). *Scheduling: Theory, Algorithms, and Systems*. Springer, Cham, Switzerland, 6 edition.

Puka, R., Skalna, I., Duda, J., and Stawowy, A. (2024). Deterministic constructive vN-NEH+ algorithm to solve permutation flow shop scheduling problem with makespan criterion. *Computers & Operations Research*, 162:106473.

Qiao, Y., Wu, N., He, Y., Li, Z., and Chen, T. (2022). Adaptive genetic algorithm for two-stage hybrid flow-shop scheduling with sequence-independent setup time and no-interruption requirement. *Expert Systems with Applications*, 208:118068.

Shabtay, D. and Gerstl, E. (2024). Coordinating scheduling and rejection decisions in a two-machine flow shop scheduling problem. *European Journal of Operational Research*, 316(3):887–898.

Shao, W., Shao, Z., and Pi, D. (2021). Effective constructive heuristics for distributed no-wait flexible flow shop scheduling problem. *Computers & Operations Research*, 136:105482.

Shao, W., Shao, Z., and Pi, D. (2023). Modelling and optimization of distributed heterogeneous hybrid flow shop lot-streaming scheduling problem. *Expert Systems with Applications*, 214:119151.

Sun, R. (2024). Randomized approximation schemes for minimizing the weighted makespan on identical parallel machines. *Journal of Combinatorial Optimization*, 47:34.

Tristan Becker, Janis Neufeld, U. B. (2025). The distributed flow shop scheduling problem with inter-factory transportation. *European Journal of Operational Research*, 322(1):39–55.

Wang, B.-T., Pan, Q.-K., Gao, L., and Li, W. (2024a). The paradoxes, accelerations and heuristics for a constrained distributed flowshop group scheduling problem. *Computers & Industrial Engineering*, 196:110465.

Wang, C., Pan, Q.-K., and Sang, H.-Y. (2024b). The cascaded flowshop joint scheduling problem: A mathematical model and population-based iterated greedy algorithm to minimize total tardiness. *Robotics and Computer-Integrated Manufacturing*, 88:102747.

Wang, Y.-J., Wang, G.-G., Tian, F.-M., Gong, D.-W., and Pedrycz, W. (2023). Solving energy-efficient fuzzy hybrid flow-shop scheduling problem at a variable machine speed using an extended NSGA-II. *Engineering Applications of Artificial Intelligence*, 121:105977.

Yuan, J., Ng, C. T., and Cheng, T. C. E. (2020). Scheduling with release dates and preemption to minimize multiple max-form objective functions. *European Journal of Operational Research*, 280(3):860–875.

Zhang, H., Lu, L., and Yuan, J. (2025). Online scheduling on an unbounded parallel-batch machine to minimize the weighted makespan. *Journal of Combinatorial Optimization*, 49(1):6.

Zheng, F., Li, N., Liu, M., and Xu, Y. (2025). Single machine lot scheduling to minimize maximum weighted completion time. *Journal of Combinatorial Optimization*, 50(1):1.

Zuo, L., Lu, L., and Zhang, L. (2026a). Single machine scheduling with a restricted weight-modifying activity to minimize the weighted makespan. *International Journal of Foundations of Computer Science*, 37(5):693–707.

Zuo, L., Zhang, J., Shi, Z., and Fan, B. (2026b). Single machine scheduling with a restricted rate-modifying activity to minimize the weighted makespan. *Discrete Applied Mathematics*, 380:89–100.